Semantic SQL via Schema Enrichment

Paper ID: 394

ABSTRACT

This paper presents LiteDBX, a lightweight database extender for answering semantic queries over relational and unstructured data. Given a database and an associated collection of unstructured data, LiteDBX enriches the schema with a small set of query-relevant attributes extracted from unstructured sources, to accurately answer semantic queries directly in SQL, without repeated online LLM invocations at query time. We formalize the schema enrichment problem under a budget on the number of added attributes, and show its intractability. We develop a data-driven approach that jointly optimizes schema enrichment and query rewriting using data-driven signals, and establish theoretical guarantees on the achievable macro-averaged F_1. We also develop an incremental maintenance framework that updates the enriched database under data and query changes, and selectively refreshes the schema using lightweight error certificates. Using real-life datasets, we empirically verify that LiteDBX is 40.3% more accurate than SOTA methods on average for semantic queries, while reducing LLM monetary cost by up to 99.9%.

1 INTRODUCTION

The need for integrating relational and unstructured data is increasingly evident for deriving comprehensive insights [49]. In applications such as clinical decision support, semantic conditions often depend on both structured records (*e.g.,* demographics, lab tests, medications) and unstructured data (*e.g.,* radiology images or physician notes) [44], to improve diagnostic accuracy, treatment planning, and prioritization of patients with greater need [61]. We refer to conditions on unstructured data as *semantic conditions*, which capture concepts (*e.g.,* clinical findings) not directly represented as structured attributes. Queries that combine standard SQL predicates with such conditions are called *semantic queries*.

Now we face a dilemma: conditions on unstructured data are either resolved at query time, incurring high cost and latency, or pre-materialized as generic features, leading to redundancy and poor alignment with query workloads. Indeed, prior work has explored several approaches. *Multi-modal databases (MMDBs)* [10, 27, 28, 33, 37, 47, 48, 54] extend SQL with natural-language predicates and interpret unstructured data online using large language models (LLMs) for cross-modal querying. *Feature stores* [15, 39, 46, 60, 62] pre-extract features from unstructured data for reuse. *Materialized views* [4, 8, 35, 50] precompute cross-domain transformations.

However, these approaches are still poorly aligned with routine production analytics in critical domains such as financial risk analysis [22] and clinical decision support [23], which demand predictable cost, low latency and auditable semantics. In many real-world workloads, queries are repeated and their meanings change slowly. The same semantic conditions on, *e.g.,* images or text, are issued again and again across routine analyses.

The problem is that MMDBs treat each query independently instead of reusing previous results; they invoke LLMs at query time to interpret conditions on unstructured data and recompute the same results across repeated queries. This incurs high monetary cost, latency and run-to-run variability, and makes decisions that are hard to inspect. Feature stores and materialized views take the opposite approach by extracting and storing large sets of generic features in advance, many of which are never used; this wastes storage, incurs redundancy, and makes it hard to adapt when queries change. As a result, these previous approaches stop short of supporting efficient, stable and interpretable analytics in production settings.

id	time (A_0)	age (A_1)	spo_2 (A_2)	RR (A_3)	CRP (A_4)	opacity (B_1)	bilateral (B_2)	atelectasis (B_3)	effusion (B_4)	notepna (B_5)	lobarconsol (B_6)	EMG (Y)
t_1	2020-12	76	89	33	140	0.78	1	0	1	–	–	Yes
t_2	2020-12	82	88	34	160	0.82	1	0	1	–	–	Yes
t_3	2020-12	69	95	20	18	0.12	0	0	0	–	–	No
t_4	2021-01	70	90	32	120	0.70	1	0	1	–	–	Yes
t_5	2021-01	67	93	26	55	0.36	1	0	1	–	–	No
t_6	2021-01	74	92	28	65	0.48	1	0	0	–	–	No
t_7	2021-01	9	95	22	10	0.12	0	1	0	–	–	No
t_8	2021-01	41	91	29	70	–	–	–	–	–	–	–
							...					
t_r	2021-02	78	90	31	110	0.72	1	0	1	–	–	Yes
							...					
t_a	2022-05	66	93	26	80	0.40	1	0	1	–	–	Yes
							...					
t_p	2023-03	71	94	24	40	0.28	0	–	0	1	1	Yes

Table 1: An enriched patient dataset with structured attributes (A_0–A_4), CXR-extracted attributes (B_1–B_6), and an emergency label (Y). Attributes B_5 and B_6 are introduced when data/queries are shifted, while B_3 is replaced.

LiteDBX. These observations motivate our approach: instead of recomputing semantic conditions for each query or materializing large and generic feature spaces, semantic results should be computed once and reused across queries. We implement this approach in LiteDBX (*Lightweight DB eXtender*), a lightweight extension for routine semantic analytics over relational and unstructured data. LiteDBX leverages the fact that, in many production workloads, query logic and semantic concepts are stable and can be *amortized once* rather than repeatedly inferred at query time.

Given a relational database $\mathcal{D}$ of schema $\mathcal{R}$, an unstructured collection $\mathcal{U}$, and a workload of semantic queries Q^S, LiteDBX selectively enriches $\mathcal{R}$ with a small set of *query-aware, human-interpretable attributes* extracted from $\mathcal{U}$, yielding an enriched schema $\mathcal{R}^E$. Semantic predicates are evaluated by rewriting queries into *standard SQL* over the enriched data, without online LLM invocation. To handle data evolution, LiteDBX incrementally maintains the enriched data and selectively refreshes the schema when semantic accuracy degrades, ensuring efficiency under updates.

Unlike MMDBs, LiteDBX avoids costly and opaque online semantic inference; unlike feature stores, it prevents over-materialization by extracting only query-relevant semantics; and unlike materialized views, it supports evolving workloads through incremental maintenance rather than fully fixed pipelines. To achieve this, LiteDBX compiles workload-level semantic interpretation into explicit relational attributes, enabling standard SQL execution with predictable performance, low cost, and auditable semantics.

Example 1: Consider a hospital dataset (Table 1) collected during the COVID-19 peak. Each patient record includes attributes time (A_0), age (A_1), SpO_2 (A_2), RR (A_3), and CRP (A_4), a binary label Y indicating emergent care, along with unstructured chest X-ray (CXR).

Clinicians need to identify patients at high risk. However, struc-

tured attributes alone are insufficient: patients with similar vitals may demonstrate very different pulmonary involvement on imaging, leading to false negatives (*e.g.,* moderate SpO_2 despite severe opacities) and false positives (*e.g.,* elevated markers without critical lung involvement) [12, 41]. This requires semantic queries that combine SQL predicates on structured data with additional conditions on CXR images, which standard SQL cannot express.

MMDB systems address this by invoking LLMs at query time to interpret such conditions, but this recomputes the same results across repeated queries, incurring high cost and latency (*e.g.,* $1.62 and 218.4s for Palimpzest on 200 rows [33]) and yielding opaque decisions. Pre-extracting features via feature stores or materialized views avoid recomputation but have to maintain large numbers of generic attributes, many of which are rarely used [4].

LiteDBX instead extracts a small set of query-relevant, interpretable attributes (*e.g.,* lung opacity extent (B_1)) to enrich the schema. Once populated, these attributes enable semantic queries to be expressed and executed as standard SQL. For new patients, attributes are computed once (via local models) and reused across queries, *e.g.,* identifying patients over 65 with low SpO_2 and severe bilateral opacity [61]. Multiple queries further share the enriched schema, enabling efficient and auditable analytics. As shown in Section 7, LiteDBX improves accuracy by 40.3% on average while reducing cost and latency by up to 99.9% and 38×, respectively. □

Challenges. Despite its appeal, LiteDBX raises several challenges. No prior work has systematically studied the following issues.

(1) Query-aware schema enrichment. Which relations in $\mathcal{R}$ should be enriched? With which features from $\mathcal{U}$? How many attributes suffice for accurate and interpretable queries under limited budgets?

(2) Accurate and low-cost query rewriting. How can semantic queries be rewritten into SQL to match or surpass state-of-the-art (SOTA) MMDB accuracy while minimizing LLM cost and latency?

(3) Hardness and robustness. Schema enrichment is NP-hard (Section 3); how can we leverage data-driven signals (*e.g.,* from LLMs) while remaining robust to noise and providing provable guarantees?

(4) Incremental maintenance. How can dataset $\mathcal{D}^E$ be efficiently maintained under updates to $\mathcal{D}$ or $\mathcal{U}$, and when should schema $\mathcal{R}^E$ be refreshed to preserve accuracy with minimal overhead?

Contributions & Organization. This paper tackles the questions.

(1) Problem formulation (Section 3). We formalize *schema enrichment for semantic queries*: given $(\mathcal{R}, \mathcal{D}, \mathcal{U}, Q^S)$, the goal is to extend $\mathcal{R}$ into $\mathcal{R}^E$ with a small set of query-relevant attributes extracted from $\mathcal{U}$, construct $\mathcal{D}^E$, and rewrite Q^S into executable SQL over $\mathcal{D}^E$ under constraints on enrichment size, supervision, and rewrite complexity. We show that the problem is NP-hard.

(2) System LiteDBX (Section 4). We present LiteDBX, a lightweight extension that compiles semantic queries Q^S into executable SQL queries Q^T over an enriched schema $\mathcal{R}^E$. LiteDBX amortizes semantic processing by computing reusable attributes once and incrementally maintaining $\mathcal{R}^E$ and Q^T under updates to $\mathcal{D}$, $\mathcal{U}$ and Q^S.

(3) Joint enrichment and rewriting (Section 5). We develop a data-driven algorithm that identifies a small set of query-relevant attributes from $\mathcal{U}$ and jointly optimizes schema enrichment and query rewriting. The resulting SQL rewrites are reusable across aligned queries, eliminating repeated online LLM inference. We provide lower bounds on the achievable average F_1 under limited supervision and bounded-SQL approximation of semantic conditions.

(4) Incremental maintenance with guarantees (Section 6). We propose an incremental framework that supports evolving data and workloads. A lightweight error certificate bounds the post-update increase in query error under $(\mathcal{R}^E, Q^T)$, enabling safe reuse of existing enrichment and rewrites. For evolving queries, LiteDBX selectively refreshes $\mathcal{R}^E$ and updates only a small set of queries.

(5) Experimental evaluation (Section 7). Using real-life datasets, we show the following: (a) LiteDBX improves accuracy over SOTA baselines in both static and dynamic settings, with 42.5% and 38.1% higher F_1 on average, respectively; (b) it reduces monetary cost by up to 99.9%; and (c) it achieves 22.6× lower latency on average.

We will discuss related work in Section 2 and summarize the novelty of the work in Section 8. We defer detailed proofs to [1].

2 BACKGROUND AND RELATED WORK

This section reviews semantic queries and prior work.

2.1 Semantic Queries

Datasets. We consider both relational and unstructured data.

Relations. Consider a database schema $\mathcal{R} = (R_1, \ldots, R_{|\mathcal{R}|})$ in which R_i is a relation schema $R_i(A_1, \ldots, A_{|R_i|})$, where each A_j is an attribute. A dataset $\mathcal{D}$ of $\mathcal{R}$ is $(D_1, \ldots, D_{|\mathcal{R}|})$, where D_i is a relation of R_i. Assume *w.l.o.g.* that each tuple $t \in D_i$ represents an entity [13].

Unstructured data. We consider a collection $\mathcal{U}$ of unstructured data, including (a) *documents* (*e.g.,* clinical notes), (b) *images* (*e.g.,* radiology scans), (c) *audio* (*e.g.,* dictations), and (d) *video* (*e.g.,* surveillance clips), represented as text, pixel arrays, waveforms and frame sequences, respectively. Each $u \in \mathcal{U}$ is treated as an opaque object at the storage level and accessed via feature extraction.

Rather than querying $\mathcal{U}$ directly at runtime, we extract semantic attributes from $\mathcal{U}$ and materialize them as typed relational attributes associated with tuples $t \in \mathcal{D}$. Each attribute represents a well-defined, human-interpretable semantic concept (*e.g.,* presence of a clinical finding, sentiment polarity, detected event) derived from the objects linked to t, enabling semantic conditions over unstructured data to be expressed and evaluated in standard SQL over an enriched schema while preserving interpretability.

We assume a mapping that associates each tuple $t \in R_i \in \mathcal{R}$ with zero or more unstructured objects $u \in \mathcal{U}$, capturing the semantic context of t through linked data (*e.g.,* clinical notes for patients or reviews for products). Such tuple–object mappings are standard in prior multimodal querying systems [27, 28, 33, 37, 47, 48].

Semantic query. Such queries extend traditional SQL by allowing users to specify *natural-language predicates* that are evaluated over both structured data $\mathcal{D}$ and associated unstructured data $\mathcal{U}$ [27, 28, 37, 47, 48]. Intuitively, such predicates capture high-level semantic intent (*e.g.,* conditions implied by text, images or videos) that cannot be expressed using standard attribute comparisons alone.

We represent a *semantic query* as a quadruple $q^S = (\Pi, \mathcal{F}, \Sigma, P_S)$, where: (1) Π is the set of projected attributes in the SELECT clause; (2) $\mathcal{F}$ is the set of joinable relation atoms $R_i \in \mathcal{R}$ in the FROM

clause; (3) Σ is the set of user-defined SQL predicates that restrict the analytical scope over $\mathcal{D}$; and (4) P_S is the set of semantic (typically natural-language) predicates referring to both $\mathcal{D}$ and $\mathcal{U}$.

Denote by $Q^S = \{q_i^S\}$ the set of user-specified semantic queries, and by $\Sigma_i(\mathcal{D})$ the set of tuples in $\mathcal{D}$ that satisfy the SQL predicates Σ of a query q_i^S. A semantic query q_i^S returns those tuples in $\mathcal{D}$ that satisfy the constraints in both Σ and P_S, *e.g.,* the analytical task in Example 1 can be represented as the semantic query q_e^S below:

```
SELECT * from Patient AS P JOIN CXR AS X ON P.id = X.id
WHERE P.time BETWEEN '2020-12' AND '2021-01' AND P.A1 ≥ 65
AND Sem(X, "the patient appears critically ill based on
clinical risk indicators and the X-ray")
```

where the two predicates (provided by users or experts [43]) form constraints over the relational attributes of Patient in Table 1, and Sem is a semantic operator applied to both Patient and image data.

2.2 Related Work

We categorize the related work as follows.

Multi-Modal DBs. This line of work extends DBMSs with native support for querying multi-modal data.

Multi-modal querying systems. Prior systems support semantic queries over multi-modal data by extending SQL with learned operators or invoking LLMs for query planning and predicate evaluation at runtime, as in LOTUS [47, 48], Palimpzest [37], CAESURA [54], ThalamusDB [27, 28], Symphony [10], FlockMTL [18] and BigQuery [21]. SemBench [33] shows that such approaches incur substantial cost and latency due to online semantic inference.

Text-only querying systems. Prior text-centric systems extend SQL with NLP or neural operators to support semantic querying over textual data, as in OpineDB [34], NeuralDB [52, 53], WannaDB [24], ZenDB [36], ELEET [55], SUQL [40] and CoddLLM [63].

Feature stores. Prior systems extract features from unstructured data and materialize them as tables to facilitate data analytics, as in RALF [60], PA-FEAT [62], FeathrPO [39], Hopsworks [15] and [46].

Materialized views. Prior systems leverage LLMs to automatically extract structured, queryable tables from document collections, as in EVAPORATE [4], TWIX [35], DocETL [50] and Doctopus [8].

This work differs from prior systems in the following. (1) *Execution model*: Unlike query-time LLM inference [21, 37, 54], LiteDBX decomposes semantic query answering into offline LLM-assisted subtasks, amortizing semantic interpretation across the workload. (2) *Execution engine*: Rather than introducing new semantic operators [18, 47, 48], LiteDBX rewrites semantic predicates into standard SQL over an enriched schema and executes them on an unmodified relational engine. (3) *Interpretability*: In contrast to embedding-based or black-box approaches [10, 37], LiteDBX yields explicit, human-interpretable predicates for transparent inspection and auditing. (4) *Cost*: By materializing workload-level semantics once as $\mathcal{R}^E$, subsequent queries execute directly over enriched data, reducing LLM cost and latency. (5) *Partial population*: Instead of materializing task-agnostic feature spaces [4, 35, 39, 60, 62], LiteDBX extracts a minimal set of query-relevant attributes. (6) *Guarantees*: LiteDBX provides theory-driven guarantees that directly lower-bound query F_1, unlike relative or heuristic bounds [27, 28, 47, 48].

Schema Enrichment. This line of work enhances relational schemas using external structured data sources to support downstream tasks, including (1) *Data discovery*, which identifies joinable or unionable tables via embeddings or semantic similarity (*e.g.,* PEXESO, AutoTUS, Starmie, Josie, Santos and Cocoa [17, 19, 20, 26, 31, 45, 65]); and (2) *ML model performance*, which improves accuracy or efficiency by enriching schemas with external data (*e.g.,* ARDA, AutoFeature, Leva, RECON and Turl [11, 16, 38, 59, 64]).

This work differs from the prior work as follows. (1) *Enrichment source*: Unlike methods that enrich schemas by joining external structured data [11, 38, 64], LiteDBX extracts semantic attributes directly from unstructured data. (2) *Workload*: Rather than targeting data discovery tasks such as join or union detection [17, 20], LiteDBX enriches schema for a workload of semantic queries.

3 PROBLEM AND COMPLEXITY

This section formulates the schema enrichment problem for answering semantic queries, and proves its intractability.

Setting. Semantic queries Q^S express analytical intent that requires joint reasoning over structured data $\mathcal{D}$ and unstructured data $\mathcal{U}$. Naively resolving such queries via online multimodal inference often incurs high computational and monetary cost, and may lead to unstable accuracy. This challenge shows up differently across two common settings. *(a) Routine workloads:* when queries and their underlying semantics are largely fixed, repeatedly performing online semantic inference is unnecessary and wasteful; instead, query rewriting and schema enrichment can be performed *once for all*, enabling stable, low-cost execution via standard SQL. *(b) Dynamic queries:* when queries change frequently, online semantic rewriting may still be required, but should be invoked selectively and guided by prior enrichment to control cost and preserve accuracy.

Our objective is to develop a lightweight database extension, LiteDBX, to support efficient execution of semantic queries using DBMS. LiteDBX extracts a small set of discriminative attributes from $\mathcal{U}$ to enrich $\mathcal{R}$ into $\mathcal{R}^E$, materializes the instance $\mathcal{D}^E$, and rewrites semantic queries into native SQL over $\mathcal{D}^E$. By limiting schema extension and amortizing semantic inference, LiteDBX achieves accuracy and interpretability while maintaining high efficiency for routine analytics over evolving multimodal data.

For routine workloads (case (a) above), subsequent queries can be expressed directly in SQL over the enriched schema $\mathcal{R}^E$. For dynamic workloads (case (b)), LiteDBX rewrites incoming semantic queries into native SQL over $\mathcal{R}^E$ to enable efficient execution.

Ground truth. Given a semantic query $q_i^S \in Q^S$, a relational dataset $\mathcal{D}$ of $\mathcal{R}$, and an unstructured dataset $\mathcal{U}$, denote by G_i the ground truth answer set of q_i^S on $\mathcal{D}$. It consists of exactly those tuples that (a) appear in $\Sigma_i(\mathcal{D})$, and (b) additionally satisfy the predicates in P_S of q_i^S by referencing $\mathcal{U}$. That is, G_i retrieves all tuples in $\mathcal{D}$ that satisfy the SQL predicates in Σ and the semantic conditions in P_S.

For instance, the ground truth of the example query q_e^S in Section 2.1 includes the Patient tuples in Example 1 for individuals aged over 65 who were admitted between December 2020 and January 2021 (from Σ), who are further identified as critically ill by the semantic predicate in P_S ("clinical risk indicators and X-ray").

Semantic selectivity. The ground truth set of q_i^S is also the subset of $\Sigma_i(\mathcal{D})$ that satisfies the semantic conditions in P_S. We associate

each tuple $t \in \Sigma_i(\mathcal{D})$ with a label $Y_i(t) \in \{0,1\}$, where $Y_i(t) = 1$ indicates that t is a positive answer to q_i^S. Denote by $\pi_i = \frac{|G_i|}{|\Sigma_i(\mathcal{D})|}$ the *semantic selectivity* of q_i^S on $\Sigma_i(\mathcal{D})$, which reflects the proportion of ground-truth tuples within the analytical scope $\Sigma_i(\mathcal{D})$ of q_i^S.

Accuracy. For each query $q_i^S \in Q^S$, denote by E_i the execution result of q_i^S over $\mathcal{D}$ and $\mathcal{U}$, *i.e.,* a set of tuples in $\mathcal{D}$ returned by the system. Denote by $\mathcal{E} = (E_1, \ldots, E_{|Q^S|})$ the execution result set of all queries in Q^S, and $\text{acc}(Q^S, \mathcal{E})$ the execution *accuracy* of the query set Q^S, measured by the macro-averaged F_1-score, *i.e.,* $\text{acc}(Q^S, \mathcal{E}) = \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} 2 \times \frac{\text{recall}_i \times \text{precision}_i}{\text{recall}_i + \text{precision}_i}$, where $\text{recall}_i = \frac{|E_i \cap G_i|}{|G_i|}$ is the ratio of tuples in E_i correctly returned to all tuples in the ground truth G_i, and $\text{precision}_i = \frac{|E_i \cap G_i|}{|E_i|}$ is the ratio of tuples in E_i correctly returned to all tuples returned in E_i.

Schema enrichment. For a relation schema $R = (\bar{A})$ of $\mathcal{R}$, where $\bar{A} = (A_1, \ldots, A_n)$ is a set of base attributes, its *enhanced schema* is $R^E = (\bar{A}, \bar{B})$, where $\bar{B} = (B_1, \ldots, B_l)$ is the set of additional attributes extracted from $\mathcal{U}$ that are relevant to the semantic queries.

For instance, one may extract attributes B_1–B_4 in Table 1 from X-ray images to respond to q_e^S in Example 1. Denote by $\text{arity}(R) = |\bar{R}|$, and $\text{arity}(\mathcal{R}) = \sum_{R \in \mathcal{R}} \text{arity}(R)$. Schema enrichment is performed under a budget β_{SE}, such that $\text{arity}(\mathcal{R}^E) - \text{arity}(\mathcal{R}) \leq \beta_{\text{SE}}$.

Query rewrite. For simplicity, we restrict SQL rewrites to unions of conjunctive queries (UCQs) [2, 58], which capture the select-project-join-union core of SQL and are expressive enough for most real-world analytical workloads. Given a semantic query $q_i^S = (\Pi, \mathcal{F}, \Sigma, P_S)$ and an enriched dataset $\mathcal{D}^E$ of schema $\mathcal{R}^E$, LiteDBX produces a rewritten query $q_i^T = (\Pi, \mathcal{F}, \Sigma, P_T)$ that preserves Π, $\mathcal{F}$ and Σ, and rewrites P_S as a β_{rew}-bounded disjunction of conjunctive predicates, *i.e.,* $P_T = \bigvee_{j=1}^{\beta_{\text{rew}}} \bar{p}_j$. Each $\bar{p}_j$ is a conjunction of comparisons $(A \oplus c)$ with $A \in \mathcal{R}^E$, constant c, and $\oplus \in \{=, \neq, <, \leq, >, \geq\}$. The resulting q_i^T is a structured decision rule over $\mathcal{D}^E$ that approximates the intent of q_i^S. Denote by $Q^T = \{q_i^T\}$ the set of rewritten queries. For instance, q_e^S in Example 1 can be rewritten as the following SQL query over the enriched schema.

```
SELECT * FROM ExtendedPatient WHERE age ≥ 65 AND time
BETWEEN '2020-12' AND '2021-01' AND SpO2 ≤ 90 AND      (1)
opacity ≥ 0.70 AND bilateral = 1 AND effusion = 1;
```

where the Π, $\mathcal{F}$, and Σ parts are inherited from q_e^S; and the semantic predicate P_S, *i.e.,* Sem in q_e^S, are rewritten as a conjunction of predicates (without disjunction), by constraining both base attributes A_0–A_2 and the enriched attributes B_1, B_2 and B_4 in Table 1 extracted from the X-ray image (see Example 5 in Section 5 for more details).

Problem. We are now ready to state *the schema enrichment problem.*

- *Input*: $Q^S, \mathcal{R}, \mathcal{D}, \mathcal{U}, \beta_{\text{SE}}$, and β_{rew} as mentioned earlier.
- *Output*: Enriched schema $\mathcal{R}^E$ of $\mathcal{R}$, extended dataset $\mathcal{D}^E$ of $\mathcal{D}$ *w.r.t.* $\mathcal{R}^E$, rewritten queries Q^T of Q^S, and query result $\mathcal{E}$ of Q^T.
- *Objective*: Find $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ that maximizes $\text{acc}(Q^S, \mathcal{E})$, subject to the schema enrichment budget $\text{arity}(\mathcal{R}^E) - \text{arity}(\mathcal{R}) \leq \beta_{\text{SE}}$.

Remark. (1) Semantic predicates P_S in Q^S do not specify which features to extract from $\mathcal{U}$; LiteDBX thus identifies a minimal set of query-relevant attributes to improve accuracy. (2) Following standard learning [56, 57], we assume that $\mathcal{U}$ induces a finite feature space, which does not affect the computational hardness of the problem [5]. (3) A semantic query q_i^S may admit multiple SQL rewrites with different predicate instantiations (i.e., possible worlds [14]); we restrict P_T to at most β_{rew} disjunctions of conjunctive predicates.

Table 2 summarizes the key notations used in this paper.

Notation	Description
$\mathcal{R}, \mathcal{R}^E, \mathcal{D}, \mathcal{D}^E$	Enriched schema $\mathcal{R}^E$ of $\mathcal{R}$, and dataset $\mathcal{D}^E$ *w.r.t.* $\mathcal{R}^E$ of $\mathcal{D}$ *w.r.t.* $\mathcal{R}$.
$\mathcal{U}$	Unstructured data associated with tuples in $\mathcal{D}$.
$q_i^S \in Q^S; \pi_i$	Semantic queries over $\mathcal{D}$ and $\mathcal{U}$; semantic selectivity of q_i^S.
$q_i^T \in Q^T$	Rewritten union-of-conjunctive-query form of each q_i^S.
G_i	Ground truth answer set of query q_i^S.
$E_i \in \mathcal{E}$	The set of tuples in $\mathcal{D}$ returned when executing $q_i^S \in Q^S$.
$\beta_{\text{SE}}, \beta_{\text{rew}}$	Budgets of enriched features, and disjunctions in the rewritten query q_i^T.

Table 2: Notations

Intractability. The decision version of the problem is to decide, given $Q^S, \mathcal{R}, \mathcal{D}, \mathcal{U}, \beta_{\text{SE}}, \beta_{\text{rew}}$ and a bound ω as input, whether there exist extensions $\mathcal{R}^E$ of $\mathcal{R}$, $\mathcal{D}^E$ of $\mathcal{D}$, and Q^T of Q^S such that $\text{acc}(Q^S, \mathcal{E}) \geq \omega$ and $\text{arity}(\mathcal{R}^E) - \text{arity}(\mathcal{R}) \leq \beta_{\text{SE}}$.

Theorem 1: *The schema enrichment problem is* NP*-hard.* □

Proof sketch: We prove NP-hardness by reduction from the NP-complete set cover problem [29] in a relaxed oracle setting where the ground-truth answer sets G_i are given as input. Given a set cover instance, we construct (1) $\mathcal{D}$ *w.r.t.* $\mathcal{R}$ encoding the elements; (2) $\mathcal{U}$ with Boolean bag-of-words features [42], each corresponding to a subset; (3) a query set $Q^S = \{q_1^S\}$; and (4) set β_{SE} to the set cover budget, $\beta_{\text{rew}} = 1$, and $\omega = 1$. We show that schema enrichment is feasible if and only if the set cover instance has a valid cover. □

4 LITEDBX: A DBMS EXTENDER

This section presents the semantic workload lifecycle in LiteDBX.

LiteDBX workload lifecycle. Figure 1 illustrates how LiteDBX handles a semantic workload over time. Its lifecycle consists of three stages: (1) the *offline stage*, where LiteDBX enriches the base schema with a small set of query-relevant attributes from $\mathcal{U}$, extends $\mathcal{D}$ accordingly, and rewrites the semantic workload into executable SQL; (2) the *online stage*, where routine semantic queries are answered by reusing existing rewritten SQL over the enriched data; and (3) the *maintenance stage*, where LiteDBX monitors whether the enriched schema/data and rewritten SQL remain reliable as data and workloads evolve, and selectively refreshes them when needed. Together, these stages enable semantic querying via offline preparation, online execution, and incremental maintenance.

We next describe each stage of the lifecycle.

(1) Offline stage: Schema enriching and query rewriting (Section 5). This stage prepares LiteDBX for a historical semantic workload Q^S under a budget by enriching base schema $\mathcal{R}$ into $\mathcal{R}^E$ with a compact set of query-relevant attributes from unstructured $\mathcal{U}$, extending $\mathcal{D}$ to $\mathcal{D}^E$, and rewriting Q^S into executable SQL queries Q^T over $\mathcal{D}^E$.

LiteDBX employs the *schema enrichment and query rewriting* (SEQR) process, which jointly selects features that (a) support reliable label inference from a small set of annotated tuples, and (b) enable accurate SQL rewrites of semantic predicates that generalize across the dataset. SEQR further provides theoretical guarantees

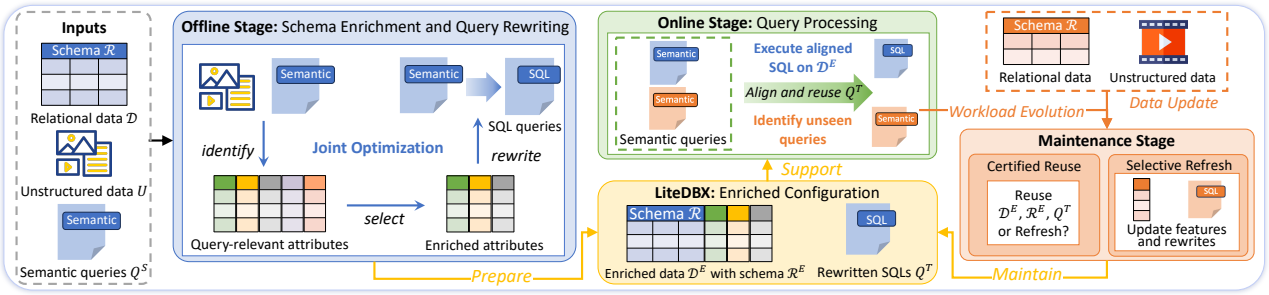


Figure 1: LiteDBX **workload lifecycle**

on the achievable average query accuracy for Q^S.

(2) Online stage: Query processing. This stage processes incoming semantic queries online after schema enrichment. For an incoming semantic query, LiteDBX checks whether an existing rewrite in Q^T can be reused under the current enriched schema $\mathcal{R}^E$ and enriched data $\mathcal{D}^E$, either by direct match or by lightweight semantic alignment with a query in Q^S. If so, the query is answered directly in SQL; otherwise, it is forwarded to the maintenance stage.

In practice, this stage handles the vast majority of routine queries, which are answered via direct reuse without additional processing.

(3) Maintenance stage: Incremental maintenance (Section 6). When data $(\mathcal{D}, \mathcal{U})$ and workloads Q^S evolve, the configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ may become unreliable: distribution shifts can reduce the accuracy of rewrites learned offline, and new or drifting queries may no longer be well captured by the current schema and rewritten workload. This stage uses a lightweight error certificate to decide whether to reuse or refresh the configuration, and selectively updates only the affected schema, data and rewrites when needed.

Example 2: Continuing with Example 1, we illustrate LiteDBX's three stages in clinical analytics; see Examples 3–5 (offline and online stages) and Examples 6–7 (maintenance stage) for more details.

(1) Offline. Clinicians issue a workload of semantic queries over patient records and associated CXR images (*e.g.*, for elderly or pediatric patients). LiteDBX jointly optimizes the workload by (a) identifying a query-relevant candidate attribute pool from CXRs using an LLM, (b) selecting a compact set of high-utility attributes under a schema budget (*e.g.*, $\{B_1, B_2, B_3, B_4\}$ in Table 1), and (c) rewriting the workload into executable SQL (*e.g.*, the one in Section 3).

(2) Online. Once the enriched schema and rewritten SQL are fixed, LiteDBX answers routine queries (*e.g.*, identifying high-risk elderly COVID-19 patients, *e.g.*, t_r in Table 1) by reusing the offline rewritten SQL with minor scope updates (*e.g.*, time windows).

(3) Maintenance. When data sources and query intents evolve (*e.g.*, age shifts or a transition to pneumonia screening), LiteDBX reuses existing rewrites whenever possible and otherwise refreshes the schema under the same budget (*e.g.*, 5). For instance, it replaces B_3 with B_5 and B_6 for the updated workload (*e.g.*, t_p), and updates the corresponding SQL rewrites accordingly. □

5 SEMANTIC SCHEMA ENRICHMENT

This section presents the *schema enrichment and query rewriting* (SEQR) process (offline stage) in LiteDBX. It introduces the offline problem and key challenges, presents the SEQR framework, and establishes its theoretical guarantees and computational complexity.

5.1 Problem and Challenge

Problem. Given relational data $\mathcal{D}$ with schema $\mathcal{R}$, unstructured data $\mathcal{U}$, and a historical semantic workload Q^S, SEQR is to (1) identify query-relevant attributes from $\mathcal{U}$; (2) select a compact enriched schema, and (3) rewrite the semantic workload into executable SQL queries for routine execution. It outputs an enriched schema $\mathcal{R}^E$, enriched data $\mathcal{D}^E$, and a rewritten SQL workload Q^T of Q^S.

Challenge. A natural approach is to employ a powerful online LLM to extract attributes from $\mathcal{U}$, enrich $\mathcal{R}$ to $\mathcal{R}^E$, extend $\mathcal{D}$ to $\mathcal{D}^E$, label a small sample of tuples, rewrite semantic queries Q^S into executable SQL queries Q^T, and execute them over $\mathcal{D}^E$. However, this process faces three challenges. (**C1**) *Cost*: per-tuple LLM inference over unstructured data is expensive, incurring monetary cost and latency. (**C2**) *Limited supervision*: only a small labeled subset can be obtained. Beyond these practical difficulties, a more subtle challenge arises. (**C3**) *Coupled optimization*: schema enrichment and query rewriting cannot be decided separately, as highly predictive attributes may not admit accurate rewrites (see below).

Joint optimization challenge. A feature may be highly discriminative but rely on a large or fragmented domain that yields very complex SQL predicates. Conversely, favoring simple rewrites may select attributes that are easy to express but too weak to capture the intended semantics. Thus, feature selection and SQL rewriting must be jointly optimized: a feature is useful only if it supports both reliable inference and a faithful, executable rewrite.

We illustrate this tension with a counterexample in the clinical scenario of Example 1. An online LLM may extract a large-domain CXR severity attribute (*e.g.*, an opacity-extent score in $\{0, \ldots, 8\}$) favored by predictive models (*e.g.*, tree-based) for fine-grained discrimination. This partitions the domain into many intervals (*e.g.*, $[0, 1), [1, 2), \ldots$), each associated with different clinical conditions (*e.g.*, SpO_2 and CRP), and induces many positive decision paths. Retaining the predictive advantage of such an attribute in SQL would require enumerating many narrow disjunctive cases across value ranges and clinical contexts, yielding a rewrite that is too complex to execute and maintain, and too fragmented to be reliably reused. This mismatch between predictive utility and rewrite compactness motivates the joint optimization in SEQR.

5.2 The SEQR Framework

Overview. SEQR addresses the three challenges via three phases: (1) *query-aware preparation*, which restricts processing to the relevant data scope and derives a query-aware attribute pool from $\mathcal{U}$ (addressing **C1**); (2) *coreset construction for compact supervision*, which expands limited annotations under budget β_{lab} into a compact supervision basis (addressing **C2**); and (3) *joint schema en-*

richment and rewriting, which evaluates candidate schemas together with their induced rewrites, selects the best schema under budget β_{SE}, and controls SQL rewriting under budget β_{rew} (addressing **C3**).

(1) Query-aware preparation. For each query $q_i^S = (\Pi, \mathcal{F}, \Sigma, P_S) \in \mathcal{Q}^S$, SEQR (a) applies the scope predicates Σ to dataset $\mathcal{D}$ to obtain the relevant tuple set $\Sigma_i(\mathcal{D})$; (b) samples a set O_i of size β_{lab} from query-scope subset $\Sigma_i(\mathcal{D})$; (c) uses query q_i^S and set O_i to derive a query-aware candidate attribute pool $\mathcal{R}_{\mathcal{U}}$ from unstructured $\mathcal{U}$ via an LLM; and (d) asks users to annotate the label $Y_i(t)$ for each tuple $t \in O_i$, providing initial supervision for refining candidate extraction and estimating query selectivity $\widehat{\pi}_i = \frac{1}{|O_i|} \sum_{t \in O_i} Y_i(t)$. The details (*e.g.,* prompts) of this step are provided in [1].

Example 3: Continuing with Example 2, we illustrate query-aware preparation in SEQR under a static workload $\mathcal{Q}^S$ during the COVID-19 peak (Dec. 2020—Feb. 2021). Let the feature budget $\beta_{\mathsf{SE}} = 5$ and labeling budget $\beta_{\mathsf{lab}} = 10$. Here $\mathcal{Q}^S = \{q_e^S, q_p^S\}$ contains semantic queries for elderly (age $\geq$ 65) and pediatric (age $\leq$ 12) patients. Tuples outside the time and age scopes are filtered by Σ (*e.g.,* t_8 in Table 1) and not materialized. For simplicity, we focus on q_e^S.

It proceeds as follows. (a) SEQR derives the SQL-filtered set $\Sigma_e(\mathcal{D})$ for q_e^S, containing elderly patients admitted between Dec. 2020 and Feb. 2021 (*e.g.,* t_1-t_6). (b) It uniformly samples a ground-truth set O_e of size $\beta_{\mathsf{lab}} = 10$ (*e.g.,* t_1-t_3). (c) Conditioned on the query and O_e, it prompts an online LLM (*e.g.,* GPT5) to propose a candidate feature pool $\mathcal{R}_{\mathcal{U}} = \{B_1, B_2, B_3, B_4, B_7, B_8, B_9\}$ from CXRs, where B_1-B_4 appear in Table 1, B_7 is the opacity-extent score ($\in \{0, \ldots, 8\}$) mentioned in Section 5.1, and B_8-B_9 capture auxiliary cues. It also populates $\mathcal{R}_{\mathcal{U}}$ for O_e and produces tentative labels (*e.g.,* {"Yes", "No", "No"} for t_1–t_3). (d) Clinicians correct a few ($\leq \beta_{\mathsf{lab}}$) errors (*e.g.,* for t_2), and SEQR uses the feedback to refine feature extraction and estimate selectivity $\widehat{\pi}_e$ (assume $\widehat{\pi}_e = 0.3$). □

Justification. It addresses challenge **C1** by restricting extraction to query-relevant tuples via scope predicates Σ, and ensuring semantics with a small labeling budget β_{lab}. This yields a query-specific attribute pool $\mathcal{R}_{\mathcal{U}}$ and a selectivity estimate $\widehat{\pi}_i$. Moreover, it requires only minimal human annotation incurred once over a small sample, and avoiding indiscriminate extraction over the full datasets.

(2) Coreset construction for compact supervision. For each query q_i^S, it expands the labeled set O_i into a coreset C_i over $\Sigma_i(\mathcal{D})$ by (a) populating candidate attributes in $\mathcal{R}_{\mathcal{U}}$ for unlabeled tuples using local LLMs, using O_i as in-context examples; (b) training a tree-based model on O_i to obtain (i) predicted labels for each tuple t in $\Sigma_i(\mathcal{D}) \setminus O_i$ with confidences $f_p(t) \in [0, 1]$ (closer to 1 (resp. 0) indicates higher confidence of being positive (resp. negative)), as well as (ii) feature-importance scores for attributes in $\mathcal{R}_{\mathcal{U}}$; (c) computing, for each $t \in \Sigma_i(\mathcal{D}) \setminus O_i$, a structural confidence $f_s(t) \in [0, 1]$:

$$f_s(t) = \frac{d^-(t)}{d^+(t) + d^-(t)},$$

where $d^+(t)$ and $d^-(t)$ are obtained by first computing the feature-importance-weighted distance from t to each labeled positive and negative tuple in O_i, respectively, and then averaging these distances within each class; and (d) combining the prediction confidence and structural confidence into a selectivity-aware score

$$s(t) = \widehat{\pi}_i f_p(t) + (1 - \widehat{\pi}_i) f_s(t),$$

which ranks tuples in $\Sigma_i(\mathcal{D}) \setminus O_i$ by $s(t)$, and constructs C_i by selecting pseudo positives from the high-score end and pseudo negatives from the low-score end, so that the positive fraction in C_i matches $\widehat{\pi}_i$. The details of this step are in [1].

Example 4: Continuing with Example 3, we illustrate the coreset construction procedure of SEQR for supervision.

It proceeds as follows. (a) For tuples in $\Sigma_e(\mathcal{D}) \setminus O_e$, SEQR populates candidate features using local models (*e.g.,* Llama3-8B), with O_e provided as in-context examples. (b) It trains a tree-based classifier (*e.g.,* XGBoost [9]) on O_e to compute feature-importance scores for attributes in $\mathcal{R}_{\mathcal{U}}$ and a prediction confidence $f_p(t)$ for each $t \in \Sigma_e(\mathcal{D}) \setminus O_e$. (c) It computes a structural confidence $f_s(t)$ based on feature-importance–weighted distances to labeled positive and negative tuples in O_e (*e.g.,* t_1 and t_3). (d) It combines the two signals as $s(t) = 0.3\, f_p(t) + 0.7\, f_s(t)$ and expands O_e into a selectivity-aware coreset C_e by selecting tuples with the highest and lowest $s(t)$ values, *e.g.,* adding t_4 (positive) and t_5, t_6 (negative). □

Justification. It addresses challenge **C2** by expanding the labeled set O_i into a selectivity-aware coreset C_i based on (a) prediction confidence, capturing how reliably a tuple can be inferred from O_i, and (b) structural confidence, capturing the similarity of an unlabeled tuple to labeled ones in O_i. It yields a compact, query-aware supervision basis without extensive annotation over $\Sigma_i(\mathcal{D})$.

(3) Joint schema enrichment and rewriting. Given the candidate attribute pool $\mathcal{R}_{\mathcal{U}}$, the coreset C_i for each $q_i^S \in \mathcal{Q}^S$, and feature-importance scores for attributes in $\mathcal{R}_{\mathcal{U}}$ (computed on C_i), SEQR obtains a global ranking of attributes in $\mathcal{R}_{\mathcal{U}}$ by aggregating their importance scores across queries in $\mathcal{Q}^S$. It then constructs candidate sets $\mathcal{R}_k$ from the top-k ranked attributes and selects the set whose rewritten SQL achieves the highest accuracy. The key idea is outlined below; more details are provided in [1].

Schema enrichment and rewriting. For each candidate attribute set $\mathcal{R}_k$, SEQR (a) extends the base schema from $\mathcal{R}$ to $\mathcal{R} \cup \mathcal{R}_k$ and populates the attributes in $\mathcal{R}_k$ for scope-satisfied tuples of each query $q_i^S \in \mathcal{Q}^S$ via local LLMs; (b) trains a tree-based model on each coreset C_i under the schema $\mathcal{R} \cup \mathcal{R}_k$ and uses its predictions to propagate labels from C_i to unlabeled tuples in $\Sigma_i(\mathcal{D}) \setminus C_i$; and (c) constructs an executable UCQ for each q_i^S by taking the disjunction of the top-β_{rew} path-converted predicates derived from high-support positive decision paths of the trained model.

Joint evaluation. For each candidate attribute set $\mathcal{R}_k$ and each query $q_i^S \in \mathcal{Q}^S$, SEQR evaluates the resulting configuration along two complementary dimensions: (a) label-propagation reliability, via a *subjective score* $\widehat{\mathcal{L}}_{\mathsf{subj}}^{(i)}$, computed as the leave-one-out (LOO) error

$$\widehat{\mathcal{L}}_{\mathsf{LOO}}^{(i)} = \frac{1}{|O_i|} \sum_{t \in O_i} \ell_{\mathsf{prop}}\Big(Y_i(t),\ \widehat{Y}_i^{\mathsf{LOO}}(t)\Big),$$

plus a penalty to account for the small size and class imbalance of O_i (see [1]). Here $Y_i(t)$ is the ground-truth label of t, $\widehat{Y}_i^{\mathsf{LOO}}(t)$ denotes the propagated label of tuple t with t excluded from the supervision set O_i, and ℓ_{prop} is a class-imbalance-aware cost-sensitive 0-1 error that weights mistakes on the minority class more heavily. (b) Rewrite fidelity, via an *objective score* $\widehat{\mathcal{L}}_{\mathsf{obj}}^{(i)}$, computed as the

Input: $\mathcal{R}$, $\mathcal{D}$, $\mathcal{U}$, $\mathcal{Q}^S$, and budgets β_{SE}, β_{lab}, β_{rew}.
Output: The enriched schema $\mathcal{R}^E$, dataset $\mathcal{D}^E$, and rewritten workload $\mathcal{Q}^T$.

1. $\{\Sigma_i(\mathcal{D})\}_{i=1}^{|\mathcal{Q}^S|} \coloneqq \mathsf{ScopeFilter}(\mathcal{Q}^S, \mathcal{D})$;
2. $(\mathcal{R}_{\mathcal{U}}, \{O_i, \widehat{\pi}_i\}_{i=1}^{|\mathcal{Q}^S|}) \coloneqq \mathsf{QueryAwarePrepare}(\mathcal{Q}^S, \{\Sigma_i(\mathcal{D})\}, \mathcal{U}, \beta_{\mathsf{lab}})$;
3. $\{C_i\}_{i=1}^{|\mathcal{Q}^S|} \coloneqq \mathsf{BuildCoresets}(\{\Sigma_i(\mathcal{D}), O_i, \widehat{\pi}_i\}, \mathcal{R}_{\mathcal{U}})$;
4. $\mathsf{FS} \coloneqq \mathsf{RankAttributes}(\{C_i\}, \mathcal{R}_{\mathcal{U}}, \beta_{\mathsf{SE}})$;
5. **for** $k \in [1, \beta_{\mathsf{SE}}]$ with $\mathcal{R}_k \coloneqq \mathsf{FS}[:k]$:
6. $\quad(\mathcal{R}^E, \mathcal{D}^E) \coloneqq \mathsf{EnrichSchema}(\mathcal{R}, \mathcal{R}_k, \mathcal{D}, \{\Sigma_i(\mathcal{D})\})$;
7. $\quad$**for** each $q_i^S \in \mathcal{Q}^S$:
8. $\quad\quad(\widehat{Y}_i, \widehat{\mathcal{L}}_{\mathsf{subj}}^{(i)}) \coloneqq \mathsf{LabelPropagation}(C_i, \Sigma_i(\mathcal{D}), \mathcal{R}^E)$;
9. $\quad\quad(q_i^T, \widehat{\mathcal{L}}_{\mathsf{obj}}^{(i)}) \coloneqq \mathsf{QueryRewrite}(\Sigma_i(\mathcal{D}), \mathcal{R}^E, \widehat{Y}_i, \beta_{\mathsf{rew}})$;
10. $\quad\mathcal{L}_{\mathsf{static}} \coloneqq \frac{1}{|\mathcal{Q}^S|}\sum_{q_i^S \in \mathcal{Q}^S}\left(\widehat{\mathcal{L}}_{\mathsf{subj}}^{(i)} + \widehat{\mathcal{L}}_{\mathsf{obj}}^{(i)}\right)$;
11. $\quad(\mathcal{R}_{\mathsf{opt}}^E, \mathcal{D}_{\mathsf{opt}}^E, \mathcal{Q}_{\mathsf{opt}}^T, \mathcal{L}_{\mathsf{opt}}) \coloneqq \mathsf{KeepBest}(\mathcal{R}^E, \mathcal{D}^E, \{q_i^T\}_{i=1}^{|\mathcal{Q}^S|}, \mathcal{L}_{\mathsf{static}})$;
12. **return** $(\mathcal{R}_{\mathsf{opt}}^E, \mathcal{D}_{\mathsf{opt}}^E, \mathcal{Q}_{\mathsf{opt}}^T)$;

Figure 2: Algorithm SEQR

mismatch between the rewritten query result under $\mathcal{R} \cup \mathcal{R}_k$ and the propagated labels on the scoped tuples of q_i^S, plus a penalty for estimating this mismatch from propagated labels under a β_{rew}-bounded rewrite. SEQR aggregates them as $\mathcal{L}_{\mathsf{static}}^{(i)} = \widehat{\mathcal{L}}_{\mathsf{subj}}^{(i)} + \widehat{\mathcal{L}}_{\mathsf{obj}}^{(i)}$, and selects $\mathcal{R}_k$ (and its induced rewrites) by minimizing $\frac{1}{|\mathcal{Q}^S|}\sum_{q_i^S \in \mathcal{Q}^S}\mathcal{L}_{\mathsf{static}}^{(i)}$.

Example 5: Continuing with Example 4, we illustrate SEQR's joint schema enrichment and query rewriting. Let the rewriting budget $\beta_{\mathsf{rew}} = 1$. Using coreset C_e, SEQR ranks candidate attributes in $\mathcal{R}_{\mathcal{U}}$, evaluates top-$k$ subsets $\mathcal{R}_k$ for $k \in [1, \beta_{\mathsf{SE}}]$, discards large-domain attributes (*e.g.,* B_7) to avoid overly coarse predicates (Section 5.1), and selects the best-performing subset. It proceeds as follows.

In the first round, SEQR trains a tree-based classifier on C_e, selects the top-1 attribute B_1 from $\mathcal{R}_{\mathcal{U}}$, extends the dataset with B_1, propagates labels, constructs a SQL rewrite under β_{rew}=1, and evaluates its F_1 (*e.g.,* > 0.6). Ultimately, it selects $\mathcal{R}^\star = \{B_1, B_2, B_3, B_4\}$ as the budget-feasible subset that maximizes the rewrite accuracy; although B_8-B_9 also rank highly, they do not further reduce the aggregated error and are thus discarded. A decision tree trained on C_e using $\mathcal{R} \cup \mathcal{R}^\star$ then yields the rewritten SQL (1) in Section 3.

Online query execution. Once the enriched schema $\mathcal{R}^E = \mathcal{R} \cup \mathcal{R}^\star$ and the rewritten SQL are fixed, LiteDBX answers aligned semantic queries (*e.g.,* identifying high-risk elderly COVID-19 patients admitted between Jan. 2021 and Feb. 2021), by reusing the SQL template above with minor updates to the scope predicates (*i.e.,* shifting the time window to Jan. 2021-Feb. 2021). It then returns the matching candidates (*e.g.,* t_r in Table 1) without re-running SEQR. □

Justification. SEQR addresses challenge **C3** along two dimensions. (1) *Implementation*, where (a) the budget β_{SE} limits the size of the enriched schema to balance semantic coverage, storage overhead and query-time cost; and (b) the UCQ restriction together with the β_{rew} budget keeps each rewrite compact, executable and interpretable, avoiding fragmented rewrites. (2) *Evaluation*, where (a) the subjective score uses LOO error to provide an unbiased estimate of label propagation reliability under limited supervision, as shown in Lemma 2 below; and (b) the objective score measures whether the same candidate attributes also support a faithful rewrite. Together, these provide us with the accuracy guarantee in Theorem 3.

Algorithm. Putting these together, we summarize SEQR in Algorithm 2. It takes as input $\mathcal{R}$, $\mathcal{D}$, $\mathcal{U}$, $\mathcal{Q}^S$, and budgets β_{SE}, β_{lab}, and β_{rew}. It returns an extended schema $\mathcal{R}^E$ of $\mathcal{R}$, an enriched dataset $\mathcal{D}^E$ of schema $\mathcal{R}^E$, and a rewritten query set $\mathcal{Q}^T$ of $\mathcal{Q}^S$.

For each query q_i^S, SEQR first derives the scoped tuple set $\Sigma_i(\mathcal{D})$, samples and labels the observed set O_i, estimates the selectivity $\widehat{\pi}_i$, and obtains the query-aware candidate attribute pool $\mathcal{R}_{\mathcal{U}}$ from $\mathcal{U}$ (lines 1-2). It then constructs the coreset C_i for each query and aggregates query-specific feature-importance scores to obtain a global ranking FS over $\mathcal{R}_{\mathcal{U}}$, where the attributes that are predictive but unlikely to contribute to compact and accurate rewrites are excluded (lines 3–4). It next performs joint schema enrichment and rewriting by iterating over candidate attribute sets $\mathcal{R}_k$, which is the set of the top-k ranked attributes (lines 5-11). In each round k, it (a) constructs the enriched schema $\mathcal{R}^E$ and enriched data $\mathcal{D}^E$ induced by $\mathcal{R}_k$ (line 6); (b) for each query q_i^S, propagates labels from C_i to scoped tuples under $\mathcal{R}^E$ and computes the subjective score (line 8); (c) rewrites each q_i^S into a β_{rew}-bounded UCQ and computes the objective score (line 9); and (d) aggregates the per-query errors and maintains the current optimal configuration (lines 10-11). Finally, it returns the best configuration $(\mathcal{R}^E, \mathcal{D}^E, \mathcal{Q}^T)$.

5.3 Theoretical Guarantee

We next present the theoretical guarantee of SEQR, beginning with the LOO estimate used in the subjective score.

Lemma 2: *Let O_i be drawn* i.i.d. *from $\Sigma_i(\mathcal{D})$ and let $\mathcal{L}_{\mathsf{prop}}^{(i)}$ denote the expected propagation error on an independently drawn tuple from $\Sigma_i(\mathcal{D})$. For any deterministic label propagation algorithm,*

$$\mathbb{E}\left[\widehat{\mathcal{L}}_{\mathsf{LOO}}^{(i)}\right] = \mathcal{L}_{\mathsf{prop}}^{(i)},$$

where the expectation $\mathbb{E}$ is over O_i and the held-out tuple. □

Proof sketch: The claim follows from the exchangeability of *i.i.d.* samples: LOO evaluation is distributionally equivalent to testing on an independent draw from $\Sigma_i(\mathcal{D})$ [7, 51] (see [1] for details). □

We now provide the workload-level guarantee of SEQR.

Theorem 3: *Given a semantic query set $\mathcal{Q}^S$, enriched data $\mathcal{D}^E$ under schema $\mathcal{R}^E$ (derived from $\mathcal{D}$ and $\mathcal{U}$), and the results $\mathcal{E}$ of executing the rewritten* SQL *workload $\mathcal{Q}^T$ produced by* SEQR*, for any $\delta \in (0,1)$ the following bound holds with probability at least $1-\delta$:*

$$\mathrm{acc}(\mathcal{Q}^S, \mathcal{E}) \;\geq\; \frac{1}{|\mathcal{Q}^S|}\sum_{q_i^S \in \mathcal{Q}^S}\Phi(\pi_i, \mathcal{L}_{\mathsf{opt}}^{(i)}),$$

$$\Phi(\pi_i, \mathcal{L}_{\mathsf{opt}}^{(i)}) = \begin{cases} \dfrac{2(\pi_i - \mathcal{L}_{\mathsf{opt}}^{(i)})}{2\pi_i - \mathcal{L}_{\mathsf{opt}}^{(i)}}, & 0 \leq \mathcal{L}_{\mathsf{opt}}^{(i)} < \pi_i, \\ 0, & \mathcal{L}_{\mathsf{opt}}^{(i)} \geq \pi_i, \end{cases}$$

where $\pi_i = \frac{|G_i|}{|\Sigma_i(\mathcal{D})|}$ is the true selectivity of q_i^S on $\Sigma_i(\mathcal{D})$, $\mathcal{L}_{\mathsf{opt}}^{(i)}$ is the aggregated error of q_i^S that yields the minimum averaged score, and $\Phi(\cdot)$ maps $(\pi_i, \mathcal{L}_{\mathsf{opt}}^{(i)})$ to a lower bound on the F1 score of q_i^S. □

Proof sketch: We prove this by (1) bounding per-query propagation (resp. rewriting) of cost-sensitive error via Lemma 2 and Hoeffding inequality [25] (resp. VC theory [57]); (2) converting the bound into the 0–1 error bound by the loss structure; (3) deriving the F_1 bound via the query selectivity and error bound; and (4) applying a union bound to obtain the accuracy guarantee on $\mathcal{Q}^S$. □

Intuitively, Theorem 3 shows that (a) SEQR is *controllable*, as minimizing each $\mathcal{L}^{(i)}_{\text{static}}$ tightens the accuracy bound; (b) with $\mathcal{L}^{(i)}_{\text{opt}}$ selected by minimizing $\mathcal{L}^{(i)}_{\text{static}}$, $\Phi\left(\pi_i, \mathcal{L}^{(i)}_{\text{opt}}\right)$ gives an F_1 bound for each $q^S_i \in Q^S$, and higher selectivity admits a higher bound under equal empirical error; and (c) the confidence parameter δ is shared across queries, making the bound workload-aware (see [1]).

Complexity. Query-aware preparation takes $O\big(|\mathcal{D}| \cdot |Q^S| + \beta_{\text{lab}} \cdot |Q^S| \cdot (c_{\text{LLM}} + c_{\text{human}})\big)$ time, where the first term derives the scope-satisfied tuple sets $\Sigma_i(\mathcal{D})$, and the second term is for candidate feature generation, human annotation, and selectivity estimation over β_{lab} tuples per query; c_{LLM} and c_{human} denote the per-tuple costs of LLM inference and human labeling, respectively.

Coreset construction needs $O(|Q^S| \cdot (c_{\text{proxy}} + c_{\text{train}} + \sigma\beta_{\text{lab}} + \sigma \log \sigma))$ time, where σ is the average size of $\Sigma_i(\mathcal{D})$, c_{proxy} is the cost of populating features in $\mathcal{R}_{\mathcal{U}}$, c_{train} is the cost of training a tree model, $\sigma\beta_{\text{lab}}$ is for computing labeled–unlabeled distances, and $\sigma \log \sigma$ is for ranking confidence scores to construct C_i.

Joint schema enrichment and rewriting needs $O(|\mathsf{FS}| \cdot |Q^S| \cdot (c_{\text{train}}\beta_{\text{lab}} + c_{\text{rank}}))$ time, where $|\mathsf{FS}|$ is the number of candidate feature sets, $c_{\text{train}} \cdot \beta_{\text{lab}}$ is for model training and LOO evaluation per coreset, and c_{rank} is for ranking decision paths by support.

6 INCREMENTAL MAINTENANCE

This section presents the incremental maintenance of LiteDBX.

Problem. The configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ produced by SEQR may become invalid in two settings: (1) *data evolution*, where updates to $\mathcal{D}$ and $\mathcal{U}$ shift the distribution of the enriched data; and (2) *query evolution*, where changes in query workload (*e.g.,* new queries) introduce semantic conditions that are not captured by the current configuration. Incremental maintenance preserves workload accuracy while avoiding full recomputation when possible.

Overview. Given updates to data and query workloads, LiteDBX follows a reuse-or-refresh pipeline: it first tests whether the current configuration remains valid via an incremental error certificate. If so, it reuses the configuration for query answering. Otherwise, it identifies affected schema, data and rewrites by constructing a refresh set and selectively updates only those components. We present our key ideas below; see [1] for more details.

Certified reuse under data evolution. When tuples in $\mathcal{D}$ or $\mathcal{U}$ are updated (*e.g.,* insertion and deletion), LiteDBX performs incremental maintenance under the pre-computed enriched schema $\mathcal{R}^E$. Specifically, it (a) updates the enriched dataset to $\mathcal{D}^E_{\text{new}}$ by populating required attributes for new tuples relevant to current queries; (b) updates coresets and re-propagates labels for each query $q^S_i \in Q^S$; and (c) computes an *incremental error certificate* $\Delta^{(i)}$ that captures both data distribution shifts and pseudo-label changes on existing tuples induced by new data. Formally, given propagated labels $\widehat{Y}_i(\cdot)$ and $\widehat{Y}^{\text{new}}_i(\cdot)$ on $\mathcal{D}^E$ and $\mathcal{D}^E_{\text{new}}$, respectively, LiteDBX evaluates the reuse reliability of $(\mathcal{R}^E, Q^T)$ over $\Sigma_i(\mathcal{D}^E)$ and $\Sigma_i(\mathcal{D}^E_{\text{new}})$ by

$$\Delta^{(i)} = \frac{|\Sigma_i(\mathcal{D}^E_{\text{new}}) \setminus \Sigma_i(\mathcal{D}^E)|}{|\Sigma_i(\mathcal{D}^E_{\text{new}})|} + \frac{\sum_{t \in \Sigma_i(\mathcal{D}^E) \cap \Sigma_i(\mathcal{D}^E_{\text{new}})} \mathbb{1}\big[\widehat{Y}_i(t) \neq \widehat{Y}^{\text{new}}_i(t)\big]}{|\Sigma_i(\mathcal{D}^E) \cap \Sigma_i(\mathcal{D}^E_{\text{new}})|}.$$

The first term quantifies the coverage shift incurred by newly introduced tuples, while the second quantifies propagated-label changes incurred on previously covered tuples after the coreset update.

LiteDBX compares $\Delta^{(i)}$ against a tolerable error bound: if satisfied, it reuses the rewrites; otherwise, it triggers selective refresh.

Example 6: Continuing with Example 2, we illustrate how LiteDBX handles data evolution after schema enrichment, starting from the enriched schema obtained in Example 5.

Suppose that beginning in May 2022, the hospital admits more "younger" elderly patients aged 65–68 (*e.g.,* t_a in Table 1). Such a distribution shift can change the relationship between base attributes (*e.g.,* SpO_2, CRP) and image-derived features. LiteDBX materializes the relevant enriched attributes (*e.g.,* B_1, B_2, and B_4) for t_a, updates the coreset (*e.g.,* including t_a), re-propagates pseudo-labels for patients admitted from Dec. 2020 to May 2022 who are not annotated by clinicians, and computes an incremental error certificate Δ that captures both the distribution shift and any induced label changes for patients admitted before May 2022. If the error certificate Δ stays within the tolerance (*e.g.,* 0.7), LiteDBX reuses the existing answer configuration. Otherwise, it refreshes the schema and SQL rewrites to reflect the new data distribution (see Example 7). □

Guarantee. We provide a post-update error bound for certified reuse.

Theorem 4: *Given $\mathcal{R}^E$, $\mathcal{D}^E$, Q^T, $\widehat{\pi}_i$, $\widehat{\pi}^E_i$ and $\mathcal{L}^{(i)}_{\text{opt}}$ for each query q^S_i produced by* SEQR *under confidence level δ, denote by $\mathcal{L}^{(i)}_{\text{update}}$ the aggregated error of q^S_i on the updated dataset $\mathcal{D}^E_{\text{new}}$ when evaluated under the same schema $\mathcal{R}^E$ and rewritten queries Q^T. Then, with probability at least $1-\delta$, the following bound holds:*

$$\mathcal{L}^{(i)}_{\text{update}} \;\leq\; \mathcal{L}^{(i)}_{\text{opt}} \;+\; \big(\Gamma^{(i)}_{\text{prop}} + \Gamma^{(i)}_{\text{rew}}\big) \cdot \Delta^{(i)}.$$

Here $\Gamma^{(i)}_{\text{prop}} = \frac{\max\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}}{\min\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}}$ and $\Gamma^{(i)}_{\text{rew}} = \frac{\max\{\widehat{\pi}^E_i, 1-\widehat{\pi}^E_i\}}{\min\{\widehat{\pi}^E_i, 1-\widehat{\pi}^E_i\}}$, where $\widehat{\pi}_i$ is the selectivity estimated on the observed set O_i, and $\widehat{\pi}^E_i$ is the fraction of tuples with positive propagated labels in $\Sigma_i(\mathcal{D}^E)$. □

Proof sketch: We prove the result by (a) noting that under fixed $\mathcal{R}^E$ and Q^T, both propagation and rewriting errors depend on changes in propagated labels; (b) bounding the error increase by decomposing updates into newly introduced tuples and label changes on existing tuples; and (c) conditioning on the same high-probability event (with probability $1-\delta$) that certifies $\mathcal{L}^{(i)}_{\text{opt}}$ (details in [1]). □

Intuitively, Theorem 4 shows that certified reuse remains controllable under data evolution. In particular, the post-update error is bounded by the original error $\mathcal{L}^{(i)}_{\text{opt}}$ plus the certified increase $(\Gamma^{(i)}_{\text{prop}} + \Gamma^{(i)}_{\text{rew}})\Delta^{(i)}$; thus, smaller $\Delta^{(i)}$ makes reuse more likely to remain valid. The factors $\Gamma^{(i)}_{\text{prop}}$ and $\Gamma^{(i)}_{\text{rew}}$ quantify the sensitivity of propagation and rewriting to class imbalance. Accordingly, LiteDBX reduces data maintenance to a direct reuse test and triggers selective refresh only when the certified increase is no longer negligible.

Selective refresh under query evolution. When new queries arrive or workload shifts introduce semantic conditions that are not captured by the current configuration, LiteDBX performs selective refresh. Specifically, it (a) identifies the affected query set by collecting newly arrived queries together with existing queries that fail the reuse test; (b) refreshes the schema for these queries by updating the candidate attribute pool and re-evaluating feature im-

portance under the schema budget β_{SE}; and (c) materializes the new attributes and refreshes the corresponding rewrites only within the affected query scopes. LiteDBX falls back to a full rerun of SEQR only when no bounded refresh can preserve the required accuracy.

Example 7: Continuing with Example 2, we illustrate how LiteDBX handles evolving query workloads after schema enrichment, starting from the enriched schema obtained in Example 6.

Dynamic queries. Assume that after the COVID-19 peak, attention shifts to general pneumonia screening, where additional unstructured sources (*e.g.,* clinical notes) become available. This yields a new semantic query q_p^S with predicate, *e.g.,* "the patient appears to have pneumonia based on clinical risk factors, X-ray and clinical notes". The query emphasizes different imaging cues: the pediatric-specific attribute B_3 becomes less informative, while new attributes B_5 = notepna (from clinical notes) and B_6 = lobarconsol (capturing lobar consolidation in CXRs) become critical.

Given a fixed schema budget, LiteDBX evaluates an expanded candidate pool, adds B_5 and B_6, and removes B_3 based on feature importance. The resulting enriched schema is $\mathcal{R}^E = \mathcal{R} \cup \{B_1, B_2, B_4, B_5, B_6\}$. The corresponding SQL rewrite is derived and reused for pneumonia-related patients (*e.g.,* t_p in Table 1).

Semantic shifts. Although rare, substantial semantic changes may occur. Tasks such as pulmonary edema assessment or heart failure diagnosis rely on different organ scales and imaging cues (*e.g.,* interstitial fluid patterns or cardiac silhouette enlargement), which are not captured by features optimized for COVID-19 or pneumonia. When the error certificate indicates that no bounded update can preserve accuracy, LiteDBX re-runs SEQR to construct a new enriched schema tailored to the updated workload. □

A unified reuse-or-refresh strategy. Taken together, LiteDBX unifies incremental maintenance into a reuse-or-refresh workflow. Given an update, it (1) incrementally maintains $\mathcal{D}^E$, updates coresets and propagated labels, and computes an error certificate per query; (2) reuses the current configuration if the certified error increase is within tolerance; (3) otherwise, performs selective refresh by identifying affected queries, updating the schema under the budget, materializing required attributes, and revising the corresponding rewrites; and (4) falls back to a full SEQR rerun only when bounded maintenance cannot preserve the required accuracy.

7 EXPERIMENTAL STUDY

Using real-life datasets, we evaluated the effectiveness, monetary cost and efficiency of LiteDBX across diverse semantic queries.

Experimental setting. We start with our settings.

Datasets. We used the same datasets as SemBench [33]; for each dataset, the base tables retain the links to unstructured modalities as provided by SemBench. Table 3 summarizes their statistics. (1) Movie is a sentiment analysis dataset of movie records with associated user reviews; (2) Wildlife is an animal identification dataset with camera-trap images; (3) E-Commerce is an online shopping dataset with product images and textual descriptions; (4) MMQA is a question-answering dataset where each tuple is grounded in a textual context, and some tuples are additionally linked to image sources; and (5) Medical is a medical dataset that includes symptom descriptions and medical images for disease analysis.

Datasets	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{R}\rvert$	$\mathcal{U}_{\text{text}}$	$\mathcal{U}_{\text{image}}$	$\lvert Q^S\rvert$
Movie [33]	1,375,738	2	1,375,738	0	4
Wildlife [33]	8,718	2	0	8,718	4
E-Commerce [33]	44,446	2	44,446	44,446	3
MMQA [33]	5,000	7	5,000	1,000	5
Medical [33]	11,112	6	1,200	10,012	5

Table 3: The tested datasets

Semantic queries. We used the same semantic queries over structured and unstructured data under static and dynamic workloads as SemBench. For static queries, we applied them to scaled datasets with the same scale factor as SemBench. For dynamic workloads, we simulated (a) data evolution by starting from 60% of the dataset and releasing an additional 20% at each update (*i.e.,* a 33% increase relative to the initial 60% base), and (b) query evolution by evaluating queries with accumulated execution context.

Evaluation metrics. As in SemBench, we evaluated LiteDBX and baselines using: (1) *Effectiveness*: the F_1 score for retrieval queries, and macro-averaged over a batched workload $\boldsymbol{Q}^S$ (Section 2.1); (2) *Monetary cost*: the cost of LLM invocations, computed from total input/output tokens based on the service pricing; and (3) *Efficiency*: end-to-end execution time for both static and dynamic workloads.

Baselines. We implemented LiteDBX in Python and compared it against: (1) LOTUS [47, 48], which supports semantic querying via sampling and proxy models. (2) Palimpzest [37], which optimizes declarative semantic queries across cost, latency and result quality. (3) ThalamusDB [27, 28], which supports semantic querying by automatically determining confidence intervals for query results under a user labeling budget. (4) BigQuery [21], an industrial system that supports semantic querying across all data modalities. (5) LOTUS+Cache, a variant of LOTUS that caches prior results, reuses the most similar cached instances when available (*e.g.,* via LLM), and otherwise falls back to the original pipeline.

Parameter settings. By default, we set (1) $\beta_{\mathsf{SE}} = 5$ for the feature budget; (2) $\beta_{\mathsf{lab}} = 50$ for the labeling budget; (3) $\beta_{\mathsf{rew}} = 5$ for the query rewrite budget; (4) Qwen3-235B-A22B (resp. Qwen3-30B-A3B-Instruct (FP8)) as the online (resp. locally-hosted) LLM; (5) Qwen3-VL-235B-A22B-Instruct (resp. Qwen3-VL-30B-A3B-Instruct (FP8)) as the online (resp. locally-hosted) VLM; and (6) $\eta = 16$ for the tolerance threshold of certificate error in dynamic query processing. Unless stated otherwise, the labeling budget of ThalamusDB is always aligned with that of LiteDBX.

Configuration. We ran experiments on a machine powered by 504 GB RAM, 104 Intel Xeon Gold 5320 CPUs @2.20 GHz, and 4 NVIDIA RTX 3090 24 GB GPUs. Unless stated otherwise, LiteDBX invokes LLMs via the vLLM inference server [32], with a parallelism degree of 20, consistent with SemBench. Each experiment was repeated three times and we report the average here.

Experimental results. We next report our findings.

Exp-1: Effectiveness in static setting. We evaluated the accuracy of LiteDBX versus baselines using F_1-score over semantic queries under routine workloads, where the schema and rewrites are obtained via SEQR over queries from [33], with semantic expressions replaced by paraphrases (also provided to LOTUS+Cache).

Accuracy vs. baselines. As shown in Table 4, we find the following.

(1) LiteDBX consistently outperforms all zero-shot baselines, achiev-

Datasets	Stats	LiteDBX			LOTUS			Palimpzest			ThalamusDB			BigQuery			LOTUS+Cache		
		F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)
Movie	max	**1.00**	**0.0003**	**6.51**	0.88	0.8520	992.96	0.88	0.8600	939.53	0.78	0.6800	2262.36	0.85	0.1037	121.24	0.86	**0.0003**	6.78
	min	**0.97**	**0.0002**	**3.59**	0.80	0.0490	67.00	0.80	0.0380	89.68	0.46	0.0780	263.27	0.60	0.0062	8.48	0.67	**0.0002**	3.88
	avg	**0.99**	**0.0003**	**5.05**	0.84	0.4505	529.98	0.84	0.4490	514.61	0.62	0.3790	1262.82	0.73	0.0550	64.86	0.77	**0.0003**	5.33
Wildlife	–	**1.00**	**0.0005**	**9.20**	0.89	0.1100	406.98	**1.00**	0.0660	146.10	0.33	0.1000	251.90	**1.00**	0.1093	168.97	**1.00**	**0.0005**	9.22
E-Comm.	max	**1.00**	**0.0006**	**8.60**	**1.00**	1.2900	1996.78	**1.00**	1.5200	890.00	0.73	1.3700	1115.02	**1.00**	1.5122	1808.68	0.92	**0.0006**	8.85
	min	**0.80**	**0.0002**	**4.59**	0.64	0.9800	881.00	0.50	0.8300	418.23	0.22	0.2880	224.67	0.30	0.2142	221.53	0.64	**0.0002**	5.70
	avg	**0.93**	**0.0004**	**6.53**	0.77	1.1400	1618.73	0.67	1.2067	673.15	0.48	0.8290	669.85	0.63	0.8272	768.80	0.76	**0.0004**	7.35
MMQA	max	**1.00**	**0.0004**	**7.90**	0.76	0.1000	138.41	**1.00**	0.0430	54.14	**1.00**	0.2400	1332.12	**1.00**	0.0152	18.90	0.86	**0.0004**	7.92
	min	**0.83**	**0.0002**	4.96	0.20	0.0750	77.07	0.76	0.0320	25.11	0.07	0.1000	370.85	0.60	0.0095	**2.46**	0.22	**0.0002**	4.99
	avg	**0.97**	**0.0003**	**6.83**	0.48	0.0844	112.03	0.92	0.0366	43.17	0.41	0.1660	901.27	0.81	0.0118	12.04	0.53	**0.0003**	6.85
Medical	max	0.84	**0.0005**	**8.31**	0.79	0.5800	683.00	**0.88**	0.7000	689.12	0.32	0.8200	2350.88	0.85	2.7591	1103.59	0.79	**0.0005**	9.31
	min	**0.51**	**0.0003**	**4.50**	0.00	0.0520	77.50	0.26	0.1023	115.85	0.19	0.0360	292.17	0.19	0.0465	59.01	0.32	**0.0003**	4.79
	avg	**0.63**	**0.0004**	**6.29**	0.37	0.2448	339.89	0.52	0.3376	313.15	0.26	0.3097	984.95	0.56	0.8639	436.22	0.49	**0.0004**	6.93

Table 4: LiteDBX **vs. baselines on routine semantic query workloads. For each dataset, we report the** max**,** min **and** avg **results over all queries in the workload (per-query results in [1]). The F_1 score, monetary cost and runtime are measured per query and aggregated. Bold (resp. underlined) entries indicate the best (resp. second-best) results.**

ing an average F_1 of 0.9, 34.8%, 14.2%, 21.4% and 27.2% higher than LOTUS, Palimpzest, BigQuery and LOTUS+Cache, respectively. This is because it (a) constructs query-aware coresets from a small set of ground-truth labels in the offline, maximizing limited supervision and mitigating LLM hallucinations; and (b) aligns queries to high-quality templates, enabling reuse of prior enrichment/rewrites without rerunning SEQR, rather than simply caching prior results with limited reusability as in LOTUS+Cache.

This highlights the gains from supervision and amortized one-time enrichment and rewrites for routine queries.

(2) LiteDBX has 115% higher F_1 than semi-supervised ThalamusDB. This is because in the offline phase LiteDBX (a) enriches the schema with needed attributes from unstructured data, enabling structured execution beyond predicate-level inference; and (b) translates semantic queries into rules over the enriched schema with interpretable attributes, reducing both false positives/negatives, *e.g.,* translating the query Q_4 on Medical into the following SQL.

```
SELECT p.patient_id FROM skin_cancer_mm AS s, patients AS p, x_ray_mm AS x
WHERE p.patient_id = s.patient_id AND p.patient_id = x.patient_id AND (
(p.age > 56 AND s.asymmetry_score > 0.67 AND x.lung_opacity_score > 0.40
    AND x.pulmonary_edema_score > 0.45)
OR (p.age <= 73 AND s.border_irregularity_score > 0.78)
OR (p.age > 73 AND x.pulmonary_edema_score > 0.20)
OR (p.age <= 73 AND s.asymmetry_score <= 0.78 AND p.smoking_history = 'Never')
    AND s.border_irregularity_score <= 0.78 AND x.lung_opacity_score > 0.30);
```

These results further justify the need of schema enrichment and validate our query translation strategy for accurate semantic querying.

We next report the impact of parameters on the accuracy.

Varying β_{SE}. We varied the feature enrichment budget β_{SE} from 1 to 9 on MMQA. As shown in Figure 3(a), the F_1 score initially increases, then stabilizes, and declines once β_{SE} exceeds 5. This is expected: (a) a larger budget allows us to include more features, improving the expressiveness of the enriched schema and thus query translation accuracy; and on the other hand, (b) including too many features can add noise and redundancy, which may in turn cause overfitting and reduce the F_1 score. We recommend setting $\beta_{\mathsf{SE}} = 5$.

Varying β_{lab}. We varied the user labeling budget β_{lab} from 10 to 90 on E-Comm in Figure 3(b). As shown there, the F_1 score consistently improves with larger β_{lab}, *e.g.,* from 0.69 to 0.987 when increasing from 10 to 90. This is because a larger labeled set provides a more accurate representation of the data distribution, yielding a higher-quality coreset, and in turn, more accurate query translation and query answering. We find that $\beta_{\mathsf{lab}} = 50$ provides a good tradeoff between the accuracy and human annotation cost.

Varying β_{rew}. We varied the rewrite budget β_{rew} from 1 to 9 on E-Comm. As shown in Figure 3(c), the F_1 score initially increases, then stabilizes, and declines when the budget becomes large (*e.g.,* > 7). This is because (a) a larger budget enables more disjunctive clauses, improving expressiveness and recall, while (b) excessive disjunctions may overfit the coreset and introduce noise that reduces accuracy. We find that $\beta_{\mathsf{rew}} = 5$ works well.

Varying LLM. We assessed LiteDBX under different LLMs on MMQA, where BigQuery is omitted as it only supports the Gemini family of LLMs. As shown in Figure 3(d), LiteDBX (a) outperforms all baselines, with 28.9%, 10.1% and 73.4% higher F_1 than LOTUS, Palimpzest and ThalamusDB, respectively; (b) is more robust to weaker LLMs, *e.g.,* its F_1 with Qwen3-30B-A3B (FP8) is 0.938, rather than 0.544 for ThalamusDB; and (c) benefits from stronger models, *e.g.,* the F_1 with Qwen3-Max is 0.984, 4.9% higher than with Qwen3-30B-A3B (FP8). These are because of (a) supervision with human-labeled data, mitigating hallucinations of weaker LLMs; (b) schema enrichment and rule-based query translation, reducing reliance on end-to-end prompting; and (c) stronger LLMs yielding higher-quality features during enrichment, improving accuracy.

Exp-2: Effectiveness in dynamic setting. We evaluated the accuracy of LiteDBX in two dynamic settings: (a) dynamic data $\mathcal{D}^E$ and (b) dynamic queries Q^S. For dynamic data, we denote each update by $\Delta\mathcal{D}^E$ with ratio γ (= $\frac{|\Delta\mathcal{D}^E|}{|\mathcal{D}^E|}$); we first run SEQR on an initial 60% of the dataset, and then progressively expand it by 33% (to 80%) and 66% (to 100%) via incremental maintenance.

Accuracy vs. baselines under dynamic $\mathcal{D}^E$. As shown in Table 5,

(1) LiteDBX consistently outperforms all baselines across evolving data at each scale. Its average F_1 is 0.895, 35%, 18.9%, 73.5%, 25.6% and 37.5% higher than LOTUS, Palimpzest, ThalamusDB, BigQuery and LOTUS+Cache, up to 46.6%, 29.3%, 67%, 36.5% and 43.2%, respectively. This is because LiteDBX extracts precise and interpretable attributes, and learns reusable SQL templates from historical routine queries, rather than relying on simple caching with limited reuse. This validates our schema enrichment and query rewriting strategy for routine queries under evolving data.

(2) We observe a slight accuracy drop as data grows: from 0 to 33%, F_1 decreases from 0.90 to 0.89, and further to 0.88 at 66%. This is expected. LiteDBX incrementally adapts the enriched schema to data updates, preserving robust query translation under evolution, and selectively reruns SEQR to refresh the schema and rewrites when updates violate the error certificate (*e.g.,* varying $\Delta\mathcal{D}^E$ on Medical). These validate that our maintenance strategy sustains accuracy while triggering adaptation only when necessary.

Datasets	γ	LiteDBX			LOTUS			Palimpzest			ThalamusDB			BigQuery			LOTUS+Cache		
		F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)
Movie	0	**0.98**	**0.0032**	120.02	0.80	0.2885	347.16	0.84	0.2635	309.36	0.73	0.3875	1438.99	0.61	0.0344	**39.28**	0.77	0.2845	281.47
	33%	**0.97**	**0**	**14.97**	0.81	0.3720	419.95	0.74	0.3510	424.00	0.69	0.3840	1318.92	0.68	0.0447	61.36	0.76	0.0800	86.58
	66%	**0.97**	**0**	**15.20**	0.78	0.4510	535.81	0.84	0.4490	514.61	0.62	0.3790	1262.82	0.73	0.0550	64.86	0.77	0.0850	111.49
Wildlife	0	**1.00**	**0.0050**	**66.56**	**1.00**	0.0650	256.89	**1.00**	0.0400	91.92	**1.00**	0.0600	175.25	**1.00**	0.0655	76.92	**1.00**	0.0650	254.74
	33%	**1.00**	**0**	**6.27**	0.86	0.0870	334.79	0.85	0.0500	120.04	0.86	0.0800	258.06	0.85	0.0873	298.40	0.89	0.0220	91.58
	66%	**1.00**	**0**	**4.39**	0.89	0.1100	406.98	**1.00**	0.0650	149.07	0.33	0.1000	296.38	**1.00**	0.1093	168.97	0.83	0.0220	89.10
E-Comm.	0	**0.97**	**0.0108**	**53.11**	0.86	0.6933	1061.27	0.71	0.7200	366.76	0.59	0.8250	626.99	0.70	0.6969	580.00	0.70	0.6967	952.61
	33%	**0.93**	**0**	**2.97**	0.81	0.9267	1293.94	0.68	0.9567	478.10	0.57	0.8400	657.34	0.68	0.7599	790.71	0.77	0.2200	308.90
	66%	**0.93**	**0**	**2.51**	0.73	1.1667	1616.04	0.67	1.2033	672.85	0.48	0.8290	669.85	0.63	0.8272	768.80	0.76	0.3933	525.37
MMQA	0	**0.98**	**0.0019**	34.24	0.52	0.0508	78.35	0.92	0.0296	37.53	0.42	0.1034	543.74	0.80	0.0072	**7.01**	0.57	0.0520	66.02
	33%	**0.97**	**0**	**2.44**	0.51	0.0648	96.31	0.84	0.0386	49.88	0.40	0.1200	668.06	0.80	0.0091	7.89	0.55	0.0258	44.66
	66%	**0.96**	**0**	**2.97**	0.50	0.0864	119.13	0.93	0.0492	66.44	0.41	0.1740	855.05	0.81	0.0118	12.04	0.53	0.0164	28.83
Medical	0	**0.59**	**0.0023**	270.05	0.23	0.1480	221.38	0.29	0.1430	**143.30**	0.20	0.2333	805.16	0.39	0.8400	415.74	0.19	0.1503	178.55
	33%	**0.60**	**0.0030**	**93.04**	0.29	0.1958	291.31	0.46	0.1850	192.34	0.21	0.2800	889.83	0.47	0.8489	464.66	0.31	0.0823	104.33
	66%	**0.59**	**0**	52.45	0.37	0.2448	338.30	0.52	0.3376	313.15	0.26	0.3097	984.95	0.56	0.8639	436.22	0.37	0.0175	**29.04**

Table 5: LiteDBX **vs. baselines on dynamic data workloads. For each dataset, we report the** avg **results over all queries in the workload (per-query results in [1]); the F_1 score, monetary cost, and runtime are computed per query and aggregated accordingly. Bold (resp. underlined) entries indicate the best (resp. second-best) results.**

Accuracy vs. baselines under dynamic Q^S. In addition to the *aligned* routine updates tested in Exp-1, we also evaluated *shifted* updates, where new queries introduce novel predicates and induce distribution shifts (not shown; see [1] for details). Under shifted updates, LiteDBX's advantage over baselines narrows (compared to Exp-1) as more unseen queries appear, *e.g.,* as the number of queries increases from 1 to 2, its F_1 drops from 1 to 0.93. When updates violate the certificate ($|Q^S| = 3$) and accuracy degrades significantly (*e.g.,* 0.93⟶0.79), LiteDBX triggers selective refresh via SEQR to update the schema and templates, restoring accuracy (*e.g.,* 0.79⟶0.94). Nevertheless, LiteDBX remains substantially cheaper and faster due to reuse and incremental maintenance (see Exp-3). These validate the dynamic maintenance under evolving query workloads.

We next evaluate the impact of parameters on the accuracy.

Varying γ. We evaluated LiteDBX under update ratio γ of $\Delta\mathcal{D}^E$ from 0 to 4/6 on Medical (the only dataset triggering refresh and the most challenging one in our experiments), where inserted (resp. deleted) tuples are sampled from tuples not yet included (resp. already included) in the current accumulated dataset.

As shown in Figure 3(e), as γ increases, the F_1 of LiteDBX first decreases, then recovers, and drops again; *e.g.,* it drops from 0.585 to 0.43 (not shown) when γ increases from 0 to 2/6, rises to 0.595 after running SEQR at 2/6, and then declines under further updates. This is because (a) the enriched schema and rewritten SQL become less reliable as more data arrives; and (b) once the error certificate is violated under sufficiently large updates, SEQR is re-run to refresh the schema/rewrites, restoring accuracy, after which further updates again introduce drift. This further validates the effectiveness of our dynamic maintenance strategy under evolving data.

Varying η. We varied the certificate tolerance η from 0 to 22. As shown in Figure 3(f), the F_1 of LiteDBX decreases as η increases; *e.g.,* it reaches the highest F_1 of 0.625 at $\eta = 0$, drops to 0.575 at $\eta = 16$, and further to 0.57 at $\eta = 22$. This is because (a) smaller η triggers more frequent refresh, better adapting to distribution shifts, while (b) larger η favors reuse and may miss substantial updates. We find $\eta = 16$ balances tradeoff between accuracy and overhead.

Exp-3: Monetary cost and efficiency. We compare LiteDBX with baselines *w.r.t.* expense and runtime under static/dynamic settings. Negligible monetary costs (below $0.0001) are reported as 0.

Expense/efficiency vs. baselines. Tables 4 and 5 show the following.

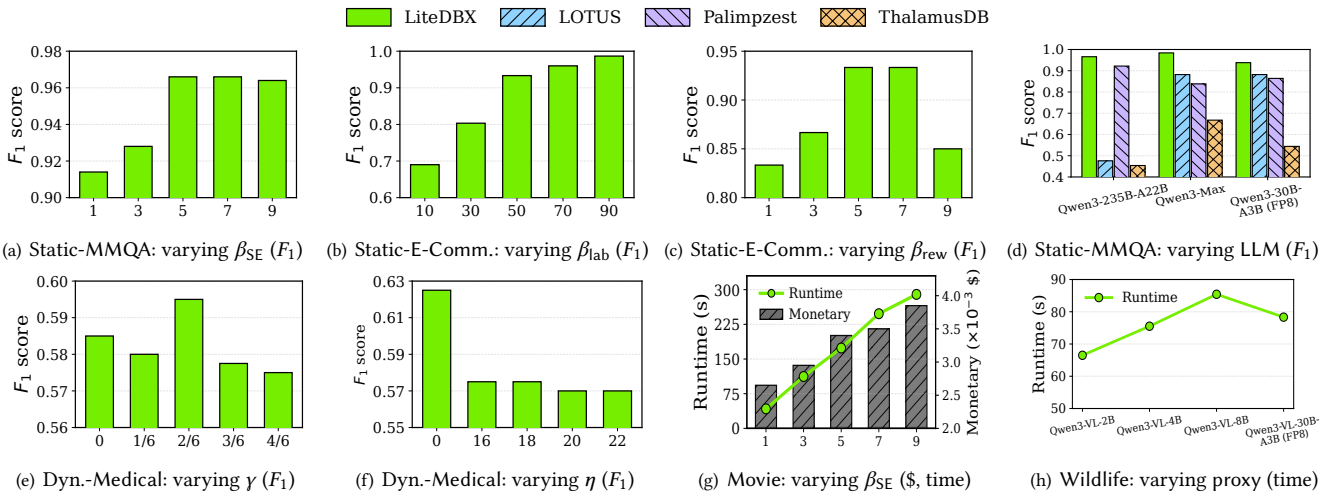
(1) In the static setting, LiteDBX is much faster and cheaper than all baselines except LOTUS+Cache. On average, its latency and monetary cost are 6.55s and $0.00039, 38× faster and 99.9% cheaper than these baselines. This is because LiteDBX (a) pushes schema enrichment, data population and query rewriting to an offline phase, enabling direct execution via rewritten SQL without online LLM calls; and (b) operates on a scope-filtered subset, reducing unnecessary computation. Thus it eliminates per-query LLM overhead and avoids full-dataset processing, yielding speedup. Although the advantage over LOTUS+Cache is smaller (9% faster with similar monetary cost), LiteDBX has much higher F_1 (Exp-1). This highlights (a) the efficiency of SEQR for routine queries by amortizing most computation in the offline phase (see dynamic setting below) and (b) the benefit of learning informative attributes from historical workloads.

(2) In the dynamic setting, LiteDBX is consistently cheaper than all baselines and faster in most cases under both dynamic data and dynamic queries (on average 98.4% and 7.1× vs. 98.4% and 3.8×). This is because (a) under data updates, although LiteDBX populates values for multiple extracted attributes via LLM proxies (rather than a single flag as in baselines), it does so only on scope-filtered tuples, significantly reducing the effective workload; and (b) under query shifts, it reruns SEQR only when the error certificate is violated, otherwise reusing the enriched schema and rewritten SQL.

We further break down the cost for dynamic data on Medical into an offline setup phase (*i.e.,* $\gamma = 0$), and a maintenance phase that updates the configuration as data evolves (*e.g.,* +33%). In the offline phase, LiteDBX is better than the baselines (114.49*s* and $0.0042 vs. 362.55*s* and $0.2730 on average), while in the maintenance phase it performs much better (23.21*s* and $0.0001 vs. 408.06*s* and $0.3059 when $\gamma = 33\%$). This is because (a) in the offline phase, cost is dominated by joint schema enrichment and query rewriting (accounting for 87.37% of runtime), together with online LLM call for labeling and feature extraction; and (b) once the schema and rewrites are fixed, the maintenance cost is dominated by value population for scope-filtered tuples via LLM proxies (accounting for 94.78% of runtime), without repeatedly invoking the online LLM.

This highlights the key advantage of LiteDBX in the incremental phase: its cost scales with the update size by reusing the enriched schema and rewrites, rather than rerunning the full pipeline as in baselines designed for static scenarios. Thus, although LiteDBX incurs a one-time setup cost, it amortizes both time and monetary cost over subsequent workloads. This validates the practicality and efficiency of schema/template reuse under evolving workloads.

We also studied how key parameters affect LiteDBX's static cost.



(a) Static-MMQA: varying β_{SE} (F_1) (b) Static-E-Comm.: varying β_{lab} (F_1) (c) Static-E-Comm.: varying β_{rew} (F_1) (d) Static-MMQA: varying LLM (F_1)

(e) Dyn.-Medical: varying γ (F_1) (f) Dyn.-Medical: varying η (F_1) (g) Movie: varying β_{SE} (\$, time) (h) Wildlife: varying proxy (time)

Figure 3: Performance evaluation

Varying β_{SE}. We varied the feature enrichment budget β_{SE} from 1 to 9 on Movie. As shown in Figure 3(g), both monetary cost and latency increase steadily with β_{SE}. This is because a larger β_{SE} (a) leads LiteDBX to invoke online LLM for suggesting more features; and (b) requires LLM proxies to populate values for more features. This said, LiteDBX remains cost-efficient when with larger budgets, *e.g.,* it incurs only \$0.00385 and 289.8s when $\beta_{\mathsf{SE}} = 9$. This validates the feasibility and efficiency of LiteDBX under various β_{SE}.

Varying proxy models. We evaluated the impact of proxy model selection on Wildlife. As offline proxies incur no monetary cost, we focus on latency. As shown in Figure 3(h), LiteDBX runs longer with stronger proxies; *e.g.,* it takes 78.3s with Qwen3-VL-30B-A3B-Instruct-FP8, 17.8% slower than with Qwen3-VL-2B-Instruct, due to higher inference cost from larger models. This said, LiteDBX remains efficient even with powerful proxies; *e.g.,* it takes 85.4s with Qwen3-VL-8B-Instruct, showing its robustness to proxy choice.

We also evaluated the scalability of LiteDBX in the static setting by varying the sizes of dataset $\mathcal{D}$, unstructured data $\mathcal{U}$, and semantic query workload $\mathcal{Q}^S$. The results show near-linear increases in latency and cost as $|\mathcal{D}|$, $|\mathcal{U}|$ and $|\mathcal{Q}^S|$ grow; see [1] for details.

Exp-4: Ablation. We also evaluated LiteDBX via following tests.

Impact of human-supervised coreset selection. We evaluated the effect of human supervision by comparing LiteDBX with a variant, $\text{LiteDBX}_{\mathsf{noSup}}$, which uses LLMs to label sampled tuples for coreset construction (not shown; see [1]). LiteDBX consistently outperforms $\text{LiteDBX}_{\mathsf{noSup}}$ across all queries, *e.g.,* 25% higher F_1 on average, up to 40.8%. This is because human supervision improves (a) feature proposal and calibration, enhancing representation quality, and (b) selectivity estimation for more accurate error control. These justify the need of supervision in coreset selection for LiteDBX.

Contribution of schema enrichment and query rewriting. We evaluated the impact of schema enrichment and query rewriting on E-Commerce by comparing LiteDBX with two variants: (a) $\text{LiteDBX}_{\mathsf{noRew}}$, which performs schema enrichment but directly queries the LLM proxy over $\mathcal{D}^E$; and (b) $\text{LiteDBX}_{\mathsf{noEnr}}$, which rewrites queries over the raw dataset $\mathcal{D}$ without enrichment (not shown). LiteDBX consistently outperforms both, with 17.2% and 30.2% higher F_1, respectively. This is because LiteDBX leverages informative attributes extracted from unstructured data, further stabilized and validated through query rewriting, rather than relying just on the LLM proxy. This validates the effectiveness of jointly optimizing schema enrichment and query rewriting.

We also evaluated the impact of (a) confidence metrics in coreset construction, (b) metrics in SEQR's static error, and (c) incremental maintenance strategies on cost and effectiveness; see [1] for details.

Summary. We find the following. (1) *Accuracy.* LiteDBX consistently outperforms SOTA baselines in both static and dynamic settings, improving F_1 by 40.3% on average. This shows the effectiveness of SEQR. (2) *Cost.* LiteDBX takes monetary cost less than baselines by up to 99.9% while having higher accuracy. (3) *Efficiency.* LiteDBX is up to 38× and 7.1× faster in query answering that baselines in the static and dynamic setting, respectively, showing the benefits of enriched schemas and query translations. (4) LLMs. LiteDBX works better with stronger LLMs: the average F_1-score with Qwen3-Max is 4.9% higher than with Qwen3-30B-A3B-Instruct (FP8), and remains robust and effective with smaller LLMs. (5) *Parameters.* We find that $\beta_{\mathsf{SE}} = 5$, $\beta_{\mathsf{lab}} = 50$, $\beta_{\mathsf{rew}} = 5$ and $\eta = 16$ balance accuracy and cost; Qwen3-235B-A22B (resp. Qwen3-VL-235B-A22B-Instruct) works well as the cloud-hosted LLM (resp. VLM), and Qwen3-30B-A3B-Instruct (FP8) (LLM) and Qwen3-VL-30B-A3B-Instruct (FP8) (VLM) as proxy models.

8 CONCLUSION

The novelty of this work consists of (1) a lightweight framework that answers routine semantic queries by rewriting them into standard SQL over relational and unstructured data; (2) a joint schema enrichment and query rewriting approach that selects query-relevant features and produces reusable SQL rewrites with accuracy guarantees; and (3) an incremental maintenance mechanism that maintains enriched schemas and SQL rewrites under evolving data and workloads. Our experiments show that LiteDBX is promising in practice.

For future work, we will develop an auto-configuration tool for LiteDBX to (a) fine-tune parameters such as the balance between accuracy and cost, and the weights of objective and subjective measures, and (b) automatically set update thresholds to trigger incremental algorithms for schema enrichment and data extraction.

REFERENCES

[1] 2026. Code, Datasets and Full Version. https://anonymous.4open.science/r/litedbx-C522/.

[2] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[3] Martin Anthony and Peter L Bartlett. 2009. *Neural Network Learning: Theoretical Foundations*. cambridge university press.

[4] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *PVLDB* (2023), 92–105.

[5] Avrim Blum and John Langford. 2003. Pac-mdl Bounds. In *COLT*. 344–357.

[6] Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K Warmuth. 1989. Learnability and the Vapnik-Chervonenkis Dimension. *JACM* 36, 4 (1989), 929–965.

[7] Prabir Burman. 1989. A Comparative Study of Ordinary Cross-Validation, V-Fold Cross-Validation and the Repeated Learning-Testing Methods. *Biometrika* 76, 3 (1989), 503–514.

[8] Chengliang Chai, Jiajun Li, Yuhao Deng, Yuanhao Zhong, Ye Yuan, Guoren Wang, and Lei Cao. 2025. Doctopus: Budget-Aware Structural Table Extraction from Unstructured Documents. *PVLDB* 18, 11 (2025), 3695–3707.

[9] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *SIGKDD*. 785–794.

[10] Zui Chen, Zihui Gu, Lei Cao, Ju Fan, Samuel Madden, and Nan Tang. 2023. Symphony: Towards Natural Language Query Answering over Multi-modal Data Lakes. In *CIDR*. 1–7.

[11] Nadiia Chepurko, Ryan Marcus, Emanuel Zgraggen, Raul Castro Fernandez, Tim Kraska, and David Karger. 2020. ARDA: Automatic Relational Data Augmentation for Machine Learning. In *PVLDB*. 2150–8097.

[12] Cosmin Citu, Oana Maria Gorun, Andrei Motoc, Ioana Mihaela Citu, Florin Gorun, and Daniel Malita. 2022. Correlation of Lung Damage on CT Scan with Laboratory Inflammatory Markers in COVID-19 Patients: A Single-Center Study from Romania. *Journal of Clinical Medicine* 11, 15 (2022), 4299.

[13] E. F. Codd. 1979. Extending the Database Relational Model to Capture More Meaning. *TODS* 4, 4 (1979), 397–434.

[14] N. Dalvi and D. Suciu. 2007. Efficient Query Evaluation on Probabilistic Databases. *VLDBJ* (2007).

[15] Javier de la Rúa Martínez, Fabio Buso, Antonios Kouzoupis, Alexandru A Ormenisan, Salman Niazi, Davit Bzhalava, Kenneth Mak, Victor Jouffrey, Mikael Ronström, Raymond Cunningham, Ralfs Zangis, Dhananjay Mukhedkar, Ayushman Khazanchi, Vladimir Vlassov, and Jim Dowling. 2024. The Hopsworks Feature Store for Machine Learning. In *Proc. ACM Manag. Data*. 135–147.

[16] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. 2022. TURL: Table Understanding through Representation Learning. *SIGMOD* 51, 1 (2022), 33–40.

[17] Yuyang Dong, Kunihiro Takeoka, Chuan Xiao, and Masafumi Oyamada. 2021. Efficient Joinable Table Discovery in Data Lakes: A High-Dimensional Similarity-Based Approach. In *ICDE*. 456–467.

[18] Anas Dorbani, Sunny Yasser, Jimmy Lin, and Amine Mhedhbi. 2025. Beyond Quacking: Deep Integration of Language Models and RAG into DuckDB. *PVLDB* (2025), 5415–5418.

[19] Mahdi Esmailoghli, Jorge-Arnulfo Quiané-Ruiz, and Ziawasch Abedjan. 2021. COCOA: COrrelation COefficient-Aware Data Augmentation. In *EDBT*. 331–336.

[20] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée Miller. 2023. Semantics-Aware Dataset Discovery from Data Lakes with Contextualized Column-Based Representation Learning. *PVLDB* (2023), 1726–1739.

[21] Sérgio Fernandes and Jorge Bernardino. 2015. What Is Bigquery?. In *IDEAS*. 202–203.

[22] José Ferreira, Fernando Almeida, and José Monteiro. 2017. Building an Effective Data Warehousing for Financial Sector. *arXiv preprint arXiv:1709.05874* (2017).

[23] Thomas Ganslandt, Sebastian Mate, Krister Helbing, Ulrich Sax, and HU Prokosch. 2011. Unlocking Data for Clinical Research–The German i2b2 Experience. *Applied clinical informatics* (2011).

[24] Benjamin Hättasch, Jan-Micha Bodensohn, Liane Vogel, Matthias Urban, and Carsten Binnig. 2023. WannaDB: Ad-hoc SQL Queries over Text Collections. In *BTW*. 157–181.

[25] Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *JASA* 58, 301 (1963), 13–30.

[26] Xuming Hu, Shen Wang, Xiao Qin, Chuan Lei, Zhengyuan Shen, Christos Faloutsos, Asterios Katsifodimos, George Karypis, Lijie Wen, and S Yu Philip. 2023. Automatic Table Union Search with Tabular Representation Learning. In *ACL*. 3786–3800.

[27] Saehan Jo and Immanuel Trummer. 2023. Demonstration of ThalamusDB: Answering Complex SQL Queries with Natural Language Predicates on Multi-Modal Data. In *SIGMOD*. 179–182.

[28] Saehan Jo and Immanuel Trummer. 2024. ThalamusDB: Approximate Query Processing on Multi-Modal Data. *Proc. ACM Manag. Data* 2, 3 (2024), 1–26.

[29] Richard M Karp. 2009. Reducibility among Combinatorial Problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 219–241.

[30] Michael J Kearns and Umesh Vazirani. 1994. *An introduction to Computational Learning Theory*. MIT press.

[31] Aamod Khatiwada, Grace Fan, Roee Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J Miller, and Mirek Riedewald. 2023. Santos: Relationship-Based Semantic Table Union Search. *SIGMOD* 1, 1 (2023), 1–25.

[32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*. 611–626.

[33] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. 2025. SemBench: A Benchmark for Semantic Query Processing Engines. *arXiv preprint arXiv:2511.01716* (2025).

[34] Yuliang Li, Aaron Xixuan Feng, Jinfeng Li, Saran Mumick, Alon Halevy, Vivian Li, and Wang-Chiew Tan. 2019. Subjective Databases. *PVLDB* (2019), 1330–1343.

[35] Yiming Lin, Mawil Hasan, Rohan Kosalge, Alvin Cheung, and Aditya G Parameswaran. 2025. Visual Template Inference for Data Extraction from Documents. *Proc. ACM Manag. Data* 3, 6 (2025), 1–27.

[36] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeigham, Aditya G Parameswaran, and Eugene Wu. 2024. Towards Accurate and Efficient Document Analytics with Large Language Models. *arXiv preprint arXiv:2405.04674* (2024).

[37] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, and Gerardo Vitagliano. 2025. Palimpzest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *CIDR*. 2.

[38] Jiabin Liu, Chengliang Chai, Yuyu Luo, Yin Lou, Jianhua Feng, and Nan Tang. 2022. Feature Augmentation with Reinforcement Learning. In *ICDE*. 3360–3372.

[39] Rui Liu, Kwanghyun Park, Fotis Psallidas, Xiaoyong Zhu, Jinghui Mo, Rathijit Sen, Matteo Interlandi, Konstantinos Karanasos, Yuanyuan Tian, and Jesús Camacho-Rodríguez. 2023. Optimizing Data Pipelines for Machine Learning in Feature Stores. *PVLDB* 16, 13 (2023), 4230–4239.

[40] Shicheng Liu, Jialiang Xu, Wesley Tjangnaka, Sina J Semnani, Chen Jie Yu, and Monica S Lam. 2024. SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models. *ACL* (2024), 4535–4555.

[41] Huanyuan Luo, Yuancheng Wang, Songqiao Liu, Ruoling Chen, Tao Chen, Yi Yang, Duolao Wang, and Shenghong Ju. 2022. Associations Between CT Pulmonary Opacity Score on Admission and Clinical Characteristics and Outcomes in Patients with COVID-19. *Internal and emergency medicine* 17, 1 (2022), 153–163.

[42] Christopher D Manning. 2009. *An Introduction to Information Retrieval*. Syngress Publishing.

[43] Himel Mondal and Muhammad Zubair. 2024. Hematocrit. *StatPearls* (2024).

[44] Emily Mu, Sarah Jabbour, Adrian V Dalca, John Guttag, Jenna Wiens, and Michael W Sjoding. 2022. Augmenting Existing Deterioration Indices with Chest Radiographs to Predict Clinical Deterioration. *PLoS One* (2022).

[45] Fatemeh Nargesian, Erkang Zhu, Ken Q Pu, and Renée J Miller. 2018. Table Union Search on Open Data. *PVLDB* 11, 7 (2018), 813–825.

[46] Laurel Orr, Atindriyo Sanyal, Xiao Ling, Karan Goel, and Megan Leszczynski. 2021. Managing ML Pipelines: Feature Stores and The Coming Wave of Embedding Ecosystems. *arXiv preprint arXiv:2108.05053* (2021).

[47] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. Lotus: Enabling Semantic Queries with LLMs over Tables of Unstructured and Structured Data. *arXiv preprint arXiv:2407.11418* (2024).

[48] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *PVLDB* (2025).

[49] Jana Sedlakova, Paola Daniore, Andrea Horn Wintsch, Markus Wolf, Mina Stanikic, Christina Haag, Chloé Sieber, Gerold Schneider, Kaspar Staub, Dominik Alois Ettlin, Oliver Grubner, Fabio Rinaldi, and Viktor von Wyl. 2023. Challenges and Best Practices for Digital Unstructured Data Enrichment in Health Research: A Systematic Narrative Review. *PLOS digital health* (2023).

[50] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *PVLDB* 18, 9 (2025), 3035–3048.

[51] Mervyn Stone. 1977. An Asymptotic Equivalence of Choice of Model by Cross-Validation and Akaike's Criterion. *J. R. Stat. Soc. Ser. B Methodol.* 39, 1 (1977), 44–47.

[52] James Thorne, Majid Yazdani, Marzieh Saeidi, Fabrizio Silvestri, Sebastian Riedel, and Alon Halevy. 2021. Database Reasoning over Text. *ACL* (2021), 3091–3104.

[53] James Thorne, Majid Yazdani, Marzieh Saeidi, Fabrizio Silvestri, Sebastian Riedel, and Alon Halevy. 2021. From Natural Language Processing to Neural Databases. In *PVLDB*, Vol. 14. 1033–1039.

[54] Matthias Urban and Carsten Binnig. 2024. CAESURA: Language Models as Multi-Modal Query Planners. In *CIDR*.

[55] Matthias Urban and Carsten Binnig. 2024. ELEET: Efficient Learned Query Execution over Text and Tables. *PVLDB* 17, 13 (2024), 4867–4880.
[56] Leslie G Valiant. 1984. A Theory of The Learnable. *CACM* 27, 11 (1984), 1134–1142.
[57] Vladimir N Vapnik. 1999. An Overview of Statistical Learning Theory. *TNNLS* 10, 5 (1999), 988–999.
[58] Moshe Y Vardi. 2016. A Theory of Regular Queries. In *PODS*. 1–9.
[59] Jiayi Wang, Chengliang Chai, Nan Tang, Jiabin Liu, and Guoliang Li. 2022. Coresets over Multiple Tables for Feature-rich and Data-efficient Machine Learning. *PVLDB* 16, 1 (2022), 64–76.
[60] Sarah Wooders, Xiangxi Mo, Amit Narang, Kevin Lin, Ion Stoica, Joseph M Hellerstein, Natacha Crooks, and Joseph E Gonzalez. 2023. RALF: Accuracy-Aware Scheduling for Feature Store Maintenance. *PVLDB* 17, 3 (2023), 563–576.
[61] Joy Tzung-yu Wu, Miguel Ángel Armengol de La Hoz, Po-Chih Kuo, Joseph Alexander Paguio, Jasper Seth Yao, Edward Christopher Dee, Wesley Yeung, Jerry Jurado, Achintya Moulick, Carmelo Milazzo, Paloma Peinado, Paula Villares, Antonio Cubillo, José Felipe Varona, Hyung-Chul Lee, Alberto Estirado, José Maria Castellano, and Leo Anthony Celi. 2022. Developing and Validating Multi-Modal Models for Mortality Prediction in COVID-19 Patients: A Multi-Center Retrospective Study. *Journal of Digital Imaging* (2022).
[62] Jianing Zhang, Zhaojing Luo, Quanqing Xu, and Meihui Zhang. 2023. PA-FEAT: Fast Feature Selection for Structured Data Via Progress-Aware Multi-Task Deep Reinforcement Learning. In *ICDE*. 394–407.
[63] Jiani Zhang, Hengrui Zhang, Rishav Chakravarti, Yiqun Hu, Patrick Ng, Asterios Katsifodimos, Huzefa Rangwala, George Karypis, and Alon Halevy. 2025. CoddLLM: Empowering Large Language Models for Data Analytics. *arXiv preprint arXiv:2502.00329* (2025).
[64] Zixuan Zhao and Raul Castro Fernandez. 2022. Leva: Boosting Machine Learning Performance with Relational Embedding Data Augmentation. In *SIGMOD*. 1504–1517.
[65] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J Miller. 2019. JOSIE: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. In *SIGMOD*. 847–864.

A PROOF OF THEOREM 1

Theorem 1: *The schema enrichment problem is* NP*-hard.* □

Proof: We show that schema enrichment is NP-hard by reduction from the NP-complete set cover problem [29]. Consider a set cover instance with an element universe $W = \{w_1, \ldots, w_n\}$, a collection of subsets $\mathcal{S} = \{S_1, \ldots, S_\ell\}$ where $S_j \subseteq W$, and a budget k. The task is to decide whether there exist at most k sets whose union covers W. We reduce set cover to a relaxed oracle version of the decision schema enrichment problem, in which the ground-truth answer sets G_i are given as part of the input.

(1) Schema $\mathcal{R}$ and instance $\mathcal{D}$: Let $\mathcal{R} = \{R(\mathrm{id})\}$ and $\mathcal{D} = \{D\}$, where D contains $n+1$ tuples with one positive tuple t_p and n negative tuples $t_1, \ldots, t_n$, each t_i corresponding to an element $w_i \in W$.

(2) Unstructured data $\mathcal{U}$: Adopting the standard bag-of-words representation [42], we construct $\mathcal{U}$ as a finite set of documents $\{d_p, d_1, \ldots, d_n\}$ over vocabulary $T = \{\tau_1, \ldots, \tau_t\}$, where d_i (resp. d_p) corresponds to the tuple t_i (resp. t_p) in $\mathcal{D}$, and each token τ_j corresponds to a subset $S_j \in \mathcal{S}$ in the set cover instance:

$$d_p = T, \qquad d_i = T \setminus \{\tau_j \mid w_i \in S_j\},$$

where d_p contains all tokens in T and each d_i excludes the tokens whose corresponding sets cover w_i. For example, as in Figure 4, the constructed document d_1 corresponding to the element w_1 does not contain tokens τ_1 and τ_l as both subsets S_1 and S_l can cover w_1.

(3) Extractable features in $\mathcal{U}$: With the finite feature space of $\mathcal{U}$, for every token τ_j, define a Boolean attribute $B_j(t) = \mathbb{1}[\tau_j \in d_t]$. That is, the Boolean feature B_j describes whether a document d_t contains the token τ_j. By construction, $B_j(t_p) = 1$ for all j, and $B_j(t_i) = 0$ iff element $w_i \in S_j$ in the set cover instance.

(4) Semantic query Q^S and ground truth G_1: Let $Q^S = \{q_1^S\}$ contain one semantic query with the ground truth $G_1 = \{t_p\}$, and in the relaxed oracle setting G_1 is given as part of the input.

(5) Budgets $\beta_{\mathsf{SE}} = k$, $\beta_{\mathsf{rew}} = 1$ and accuracy threshold $\omega = 1$.

The reduction can be constructed in PTIME. Since Q^S contains a single query, $\mathrm{acc}(Q^S, \mathcal{E})$ coincides with its F_1-score.

We now show that there exist extensions $\mathcal{R}^E$ of $\mathcal{R}$, $\mathcal{D}^E$ of $\mathcal{D}$ with at most β_{SE} attributes (*i.e.,* $\mathrm{arity}(\mathcal{R}^E) - \mathrm{arity}(\mathcal{R}) \le \beta_{\mathsf{SE}}$) and rewritten queries Q^T for Q^S with at most β_{rew} disjunctions of conjunctive predicates such that $\mathrm{acc}(Q^S, \mathcal{E}) \ge 1$, if and only if there exists at most k sets whose union covers W in the set cover.

$\Rightarrow$ Assume that there exist extensions $\mathcal{R}^E$ of $\mathcal{R}$ and $\mathcal{D}^E$ of $\mathcal{D}$ with at most $\beta_{\mathsf{SE}} = k$ enriched attributes, together with rewritten queries $Q^T = \{q_1^T\}$ for $Q^S = \{q_1^S\}$, such that $\mathrm{acc}(Q^S, \mathcal{E}) \ge 1$. Then q_1^T executed on $\mathcal{D}^E$ *w.r.t.* $\mathcal{R}^E$ must return exactly $\mathcal{E} = \{t_p\}$. By construction, $B_j(t_p) = 1$ for every extracted attribute B_j from $\mathcal{U}$; hence any q_1^T that includes at least one predicate $B_j = 1$ must return t_p. For a negative tuple t_i to be excluded by q_1^T, there must exist some enriched attribute B_j in $\mathcal{R}^E$ that appears in the predicate of q_1^T such that $B_j(t_i) = 0$, which holds precisely when $w_i \in S_j$. Thus, every $w_i \in W$ must belong to at least one set S_j associated with an enriched attribute in $\mathcal{R}^E$ that appears in q_1^T. Since $\mathcal{R}^E$ contains at most $\beta_{\mathsf{SE}} = k$ enriched attributes $\bar{B}$ from $\mathcal{U}$, the subsets in $\mathcal{S}$ corresponding to $\bar{B}$ form a set cover of the universe W.

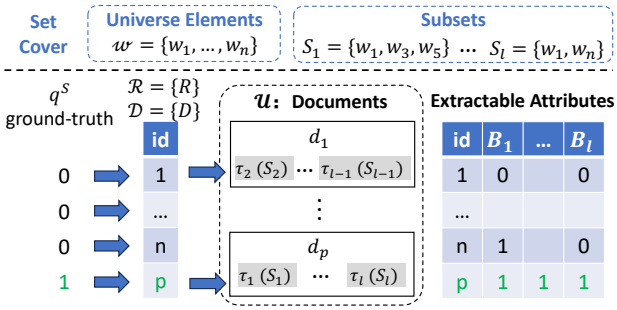


Figure 4: Reduction example

$\Leftarrow$ Conversely, assume that the set cover instance has a feasible solution $\{S_{j_1}, \ldots, S_{j_k}\}$ of size at most $k = \beta_{\mathsf{SE}}$ whose union is W. We construct $\mathcal{R}^E$ by adding the attributes $\{B_{j_1}, \ldots, B_{j_k}\}$ extracted from $\mathcal{U}$ into $\mathcal{R}$ and obtain q_1^T with predicates $B = 1$ for all $B \in \{B_{j_1}, \ldots, B_{j_k}\}$. By construction, $B(t_p) = 1$ for every extracted attribute B from $\mathcal{U}$; hence any rewritten query q_1^T with a predicate $B = 1$ must return t_p. For a negative tuple t_i, the fact that w_i belongs to one of the selected sets S_{j_r} implies $B_{j_r}(t_i) = 0$, and therefore t_i is excluded by q_1^T. Thus q_1^T executed on $\mathcal{D}^E$ returns exactly $\mathcal{E} = \{t_p\}$. Since at most $k = \beta_{\mathsf{SE}}$ enriched attributes are added, the schema enrichment instance yields feasible extensions $\mathcal{R}^E$ of $\mathcal{R}$, $\mathcal{D}^E$ of $\mathcal{D}$, and a rewritten query set Q^T with $\mathrm{acc}(Q^S, \mathcal{E}) \ge 1$. □

B DETAILS FOR SEQR

This section gives details for the SEQR process in Section 5.

Query-aware preparation. It processes in the following steps.

Step 1: Scoped tuple retrieval and initial extraction. For each query $q_i^S = (\Pi, \mathcal{F}, \Sigma_i, P_S)$, it first applies the scope predicate Σ_i on $\mathcal{D}$ to obtain the scoped tuple set $\Sigma_i(\mathcal{D})$, and then samples a subset O_i of size β_{lab} from $\Sigma_i(\mathcal{D})$. It next prompts an online LLM with (a) the semantic condition P_S in q_i^S and (b) the tuples in O_i together with their associated unstructured contents in $\mathcal{U}$, to obtain an initial candidate attribute pool, attribute values on tuples in O_i, and pseudo labels for these tuples, as illustrated in Prompt-1.

LLM Prompt-1: Query-Aware Attribute Extraction.

= **TASK OVERVIEW** =
You are given a semantic query, a sampled set of tuples from its scoped data subset, and the unstructured contents associated with these tuples. Your task is to analyze the semantic condition in the query, inspect the sampled tuples together with their linked unstructured contents, and produce an *initial* query-aware candidate attribute pool for schema enrichment.

= **OBJECTIVE** =
Infer a small set of *explicit, stable, and human-interpretable* attributes that can be extracted from the unstructured source alone and are potentially useful for evaluating the semantic condition. For each sampled tuple, also return: (1) tentative values of the extracted attributes, and (2) a pseudo label indicating whether the tuple appears to satisfy the semantic condition.

= **INPUT SPECIFICATION** =
The input consists of the following parts:
(1) `semantic_condition`: the semantic condition in the query.
(2) `sampled_tuples`: a sampled subset of tuples from the scoped data relevant to the query.
(3) `unstructured_contents`: the unstructured contents associated with the sampled tuples.
(4) `feature_budget`: the maximum number of candidate attributes to propose.

= **INPUT EXAMPLE** =
```
semantic_condition:
"The patient appears critically ill based on clinical indicators and CXR."
sampled_tuples:
[{"tuple_id":"t1","time":"2020-12","age":76,"SpO2":89,"CRP":140},
 {"tuple_id":"t2","time":"2020-12","age":82,"SpO2":88,"CRP":160},
```

```
 {"tuple_id":"t3","time":"2020-12","age":69,"SpO2":95,"CRP":18}]
unstructured_contents:
{"t1":"CXR shows diffuse bilateral opacity with pleural effusion.",
 "t2":"CXR shows extensive bilateral infiltrates and severe opacity.",
 "t3":"CXR shows mild focal change without bilateral involvement."}
feature_budget: 5
```

= **INSTRUCTIONS (STEP-BY-STEP)** =

- Read the semantic condition and determine what evidence in the unstructured content would be relevant to deciding whether a tuple satisfies this condition.
- Examine the sampled tuples together with their associated unstructured contents, and infer pseudo labels indicating whether each tuple appears to satisfy the semantic condition.
- Based on the semantic condition and the sampled tuples, identify a small set of candidate attributes that are: (a) directly extractable from the unstructured source alone, (b) stable across tuples, (c) useful for distinguishing tuples that likely satisfy the semantic condition from those that do not, and (d) suitable for downstream filtering, ranking, or query rewriting.
- For each candidate attribute, provide: (a) a concise snake_case attribute name, (b) its type, restricted to `bool`, `int`, or `float`, and (c) a precise extraction instruction that can later be used to populate this attribute for a single tuple.
- For each sampled tuple, return the tentative value of every proposed attribute, together with the pseudo label inferred for this tuple.
- Propose no more than the specified feature budget, and avoid free-text, categorical, or high-cardinality string attributes.

= **OUTPUT SPECIFICATION** =

Return a valid JSON object with the following fields:

```
{
  "candidate_features": [
    {"target_col": "feature_name",
     "feature_type": "bool | int | float",
     "extraction_prompt": "instruction for extracting this feature from one
       tuple"}],
  "tuple_annotations": [
    {"tuple_id": "identifier of a sampled tuple",
     "pseudo_label": <boolean>,
     "feature_values": {"feature_name_1": <value>}}]
}
```

= **OUTPUT EXAMPLE** =

```
{
  "candidate_features": [
    {"target_col": "opacity",
     "feature_type": "float",
     "extraction_prompt": "Estimate the extent of lung opacity from the CXR
       as a number between 0 and 1."}],
  "tuple_annotations": [
    {"tuple_id": "t1",
     "pseudo_label": true,
     "feature_values": {"opacity": 0.78}}]
}
```

Step 2: Refinement with annotated feedback. After the initial extraction, users annotate the tuples in O_i to provide ground-truth labels. It then feeds these corrected labels, together with the candidate attributes and values returned in Step 1, back to the LLM, such that it can refine the candidate attribute pool $\mathcal{R}_{\mathcal{U}}$ by removing or updating unsuitable attributes and refreshing their materialized values on O_i, following Prompt-2. The corrected labels are also used to estimate the selectivity $\widehat{\pi}_i = \frac{1}{|O_i|} \sum_{t \in O_i} Y_i(t)$.

LLM Prompt-2: Attribute Refinement with Annotated Feedback.

= **TASK OVERVIEW** =

You are given a semantic query, a sampled set of tuples from its scoped data subset, the unstructured contents associated with these tuples, and a previously proposed candidate attribute pool with tentative attribute values and pseudo labels indicating whether the tuple appears to satisfy the semantic condition. Users have now provided corrected labels for the sampled tuples. Your task is to refine the previously proposed attributes using this annotated feedback, so that the retained attributes better align with the semantic condition and the labeled evidence.

= **OBJECTIVE** =

Update the previously proposed candidate attribute pool by removing, revising, or retaining attributes as needed. Use the corrected labels as feedback on the initial extraction results. For each retained attribute, also return refreshed attribute values on the sampled tuples that are consistent with the refined attribute definitions.

= **INPUT SPECIFICATION** =

The input consists of the following parts:
(1) `semantic_condition`: the semantic condition in the query.
(2) `sampled_tuples`: the sampled subset of tuples used in the initial extraction step.
(3) `unstructured_contents`: the unstructured contents associated with the sampled tuples.
(4) `initial_candidate_features`: the previously proposed candidate attributes.
(5) `initial_tuple_annotations`: the tentative attribute values and pseudo labels previously returned for the sampled tuples.
(6) `corrected_labels`: the ground-truth labels provided by users for the sampled tuples.

= **INPUT EXAMPLE** =

```
semantic_condition:
"The patient appears critically ill based on clinical indicators and CXR."
sampled_tuples:
[{"tuple_id":"t2","time":"2020-12","age":82,"SpO2":88,"CRP":160}]
unstructured_contents:
{"t2":"CXR shows extensive bilateral infiltrates and severe opacity."}
initial_candidate_features:
[{"target_col":"opacity","feature_type":"float",
  "extraction_prompt":"Estimate the extent of lung opacity from the CXR as a
       number between 0 and 1."}]
initial_tuple_annotations:
[{"tuple_id":"t2","pseudo_label":false,
  "feature_values":{"opacity":0.82}}]
corrected_labels:
{"t2":true}
```

= **INSTRUCTIONS (STEP-BY-STEP)** =

- Read the semantic condition, the previously proposed candidate attributes, the initial tuple annotations, and the corrected labels, and identify which parts of the initial extraction appear consistent or inconsistent with the annotated evidence.
- Compare the initial pseudo labels with the corrected labels, and use the disagreements as feedback for refining the previously proposed attributes.
- Remove attributes that appear irrelevant, unstable, redundant, or poorly aligned with the corrected labels.
- Revise retained attributes when needed, including their names, definitions, or extraction instructions, so that they better capture evidence relevant to the semantic condition.
- Refresh the attribute values of the retained attributes on the sampled tuples using the refined definitions.
- Keep the refined attribute set compact, interpretable, and directly extractable from the unstructured source alone. Avoid free-text, categorical, or high-cardinality string attributes.

= **OUTPUT SPECIFICATION** =

Return a valid JSON object with the following fields:

```
{
  "refined_candidate_features": [
    {"target_col": "feature_name",
     "feature_type": "bool | int | float",
     "extraction_prompt": "instruction for extracting this feature from one
       tuple"}],
  "refined_tuple_annotations": [
    {"tuple_id": "identifier of a sampled tuple",
     "feature_values": {"feature_name_1": <value>}}]
}
```

= **OUTPUT EXAMPLE** =

```
{
  "refined_candidate_features": [
    {"target_col": "opacity",
     "feature_type": "float",
     "extraction_prompt": "Estimate the extent of lung opacity from the CXR
       as a number between 0 and 1."}],
  "refined_tuple_annotations": [
    {"tuple_id": "t2",
     "feature_values": {"opacity": 0.82}}]
}
```

Coreset construction for supervision. It proceeds as follows.

Step 1: Feature population. To materialize candidate attributes in $\mathcal{R}_{\mathcal{U}}$ for tuples in $\Sigma_i(\mathcal{D}) \setminus O_i$, SEQR prompts low-cost locally deployed models (*e.g.,* Llama3-8B) with tuples in O_i as in-context examples, as illustrated in Prompt-3. When multiple local models are used, SEQR accepts the majority output when they reach consensus; otherwise, it falls back to a stronger cloud-hosted LLM.

LLM Prompt-3: Attribute Population with In-Context Examples.

= **TASK OVERVIEW** =
You are given a set of attributes with extraction instructions, a set of example tuples with associated unstructured contents and known attribute values, and a set of new tuples whose attribute values need to be populated. Your task is to infer the values of all target attributes for each new tuple by following the extraction instructions and using the provided examples as guidance.

= **OBJECTIVE** =
Populate the values of a *set* of candidate attributes for a *set* of target tuples. The returned values must follow the required attribute types and be jointly consistent with the extraction instructions, the in-context examples, and the unstructured contents of the target tuples.

= **INPUT SPECIFICATION** =
The input consists of the following parts:
(1) `target_attributes`: the set of attributes to be populated, where each attribute includes its name, type, and extraction instruction.
(2) `in_context_examples`: a small set of example tuples, each with its unstructured content and the known values of the target attributes.
(3) `target_tuples`: the tuples whose attribute values need to be populated.
(4) `target_contents`: the unstructured contents associated with the target tuples.

= **INPUT EXAMPLE** =
```
target_attributes:
[{"target_col":"opacity","feature_type":"float",
  "extraction_prompt":"Estimate the extent of lung opacity from the CXR as a
       number between 0 and 1."},
 {"target_col":"bilateral","feature_type":"bool",
  "extraction_prompt":"Does the CXR show bilateral pulmonary involvement?
      Respond with True or False."},
 {"target_col":"effusion","feature_type":"bool",
  "extraction_prompt":"Does the CXR show pleural effusion? Respond with True
       or False."}]
in_context_examples:
[{"tuple_id":"t1",
  "content":"CXR shows diffuse bilateral opacity with pleural effusion.",
  "feature_values":{"opacity":0.78,"bilateral":true,"effusion":true}},
 {"tuple_id":"t3",
  "content":"CXR shows mild focal change without bilateral involvement.",
  "feature_values":{"opacity":0.12,"bilateral":false,"effusion":false}}]
target_tuples:
[{"tuple_id":"t4","time":"2021-01","age":70,"SpO2":90,"CRP":120}]
target_contents:
{"t4":"CXR shows marked bilateral opacity with pleural effusion."}
```

= **INSTRUCTIONS (STEP-BY-STEP)** =
- Read all target attributes, their types, and their extraction instructions carefully.
- Examine the in-context examples and use them to understand how the target attributes should be inferred from unstructured content.
- For each target tuple, inspect its associated unstructured content and infer the values of all target attributes by applying the same extraction logic jointly.
- Ensure that each returned value matches the required attribute type: return `true` or `false` for `bool`, an integer for `int`, and a numeric score for `float`.
- Return only the populated values for the requested attributes of each target tuple. Do not include explanations, comments, or additional text.

= **OUTPUT SPECIFICATION** =
Return a valid JSON object with the following field:
```
{
  "tuple_annotations": [
    {"tuple_id": "identifier of a target tuple",
     "feature_values": {"target_col_1": <value>, "target_col_2": <value>}
     }]
}
```

= **OUTPUT EXAMPLE** =
```
{
  "tuple_annotations": [
    {"tuple_id": "t4",
     "feature_values": {"opacity": 0.70, "bilateral": true, "effusion": true}
     }]
}
```

Step 2: Prediction confidence. Using the populated attributes in $\mathcal{R} \cup \mathcal{R}_{\mathcal{U}}$, SEQR trains a tree-based classifier on O_i. For each unlabeled tuple $t \in \Sigma_i(\mathcal{D}) \setminus O_i$, the classifier outputs (a) a prediction confidence $f_p(t) \in [0, 1]$ and (b) feature-importance scores over attributes in $\mathcal{R}_{\mathcal{U}}$. Here $f_p(t)$ measures how strongly the classifier predicts t to satisfy the query semantics under the current supervision.

Step 3: Structural confidence. For each unlabeled tuple $t \in \Sigma_i(\mathcal{D}) \setminus O_i$, SEQR computes a structural confidence $f_s(t) \in [0, 1]$ by comparing t with labeled tuples in O_i. It first computes the feature-importance-weighted L2 distance from t to each labeled positive and negative tuple in O_i, and then averages these distances within each class to obtain $d^+(t)$ and $d^-(t)$, respectively. The structural confidence is defined as

$$f_s(t) = \frac{d^-(t)}{d^+(t) + d^-(t)}.$$

Thus, $f_s(t)$ is larger (resp. smaller) when t is structurally closer to labeled positives (resp. negatives).

Step 4: Selectivity-aware coreset construction. Given the prediction confidence $f_p(t)$ and structural confidence $f_s(t)$, SEQR combines them into a selectivity-aware score

$$s(t) = \widehat{\pi}_i f_p(t) + (1 - \widehat{\pi}_i) f_s(t),$$

where $\widehat{\pi}_i$ is the estimated selectivity from O_i. It then ranks tuples in $\Sigma_i(\mathcal{D}) \setminus O_i$ by $s(t)$ in descending order and constructs C_i as follows: it (a) fixes the coreset size under a budget chosen with respect to the data scale and labeling budget; (b) selects pseudo positives with the highest scores so that the positive fraction in C_i matches $\widehat{\pi}_i$; and (c) selects pseudo negatives with the lowest scores to fill C_i.

Joint evaluation in schema enrichment and rewriting. It consists of the following steps.

Step 1: Candidate attribute ranking and construction. Given the candidate attributes $\mathcal{R}_{\mathcal{U}}$, the coreset C_i for each query $q_i^S \in Q^S$, and the feature-importance score $w_i(A)$ of each attribute $A \in \mathcal{R}_{\mathcal{U}}$ computed on C_i, it aggregates query-specific scores into a global score

$$w(A) = \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} w_i(A),$$

which ranks attributes in $\mathcal{R}_{\mathcal{U}}$ by $w(A)$, and constructs candidate attribute sets $\mathcal{R}_k$ from the top-k ranked attributes for $k \in [1, \beta_{\mathrm{SE}}]$.

Step 2: Label propagation. For each candidate attribute set $\mathcal{R}_k$, it constructs the enriched schema $\mathcal{R}^E = \mathcal{R} \cup \mathcal{R}_k$ and materializes the selected attributes in $\mathcal{R}_k$ for scope-filtered tuples in $\Sigma_i(\mathcal{D})$, following the same feature-population procedure as in Step 1 of coreset construction, to obtain the corresponding enriched data $\mathcal{D}^E$. For each query q_i^S, it then trains a tree-based model on the coreset C_i under $\mathcal{R}^E$, and applies the trained model to tuples in the scoped enriched set $\Sigma_i(\mathcal{D}^E) \setminus C_i$. The predicted labels are used as propagated labels for these tuples, while the labels of tuples in C_i remain fixed. As a result, each tuple in $\Sigma_i(\mathcal{D}^E)$ is associated with either a user-annotated label or a propagated label, and the propagated labels outside C_i may change across candidate schemas.

Step 3: Subjective score computation. For each query q_i^S, SEQR evaluates label-propagation reliability under the candidate schema by first computing a leave-one-out (LOO) error on the observed set O_i, and then adding a confidence-based penalty.

LOO *error.* For each tuple $t \in O_i$, denote by $\widehat{Y}_i^{\mathrm{LOO}}(t)$ the propagated label of t obtained by performing label propagation with $O_i \setminus t$ as the seed set. Given the ground-truth label $Y_i(t)$ of t, the LOO error is:

$$\widehat{\mathcal{L}}_{\mathrm{LOO}}^{(i)} = \frac{1}{|O_i|} \sum_{t \in O_i} \ell_{\mathrm{prop}}\left(Y_i(t),\ \widehat{Y}_i^{\mathrm{LOO}}(t)\right).$$

Here $\ell_{\text{prop}}(\cdot)$ is a class-imbalance-aware 0–1 error, defined as:

$$\ell_{\text{prop}}\big(Y_i(t), \widehat{Y}_i^{\text{LOO}}(t)\big) = \begin{cases} \frac{\max\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}}{\widehat{\pi}_i}, & Y_i(t)=1,\ \widehat{Y}_i^{\text{LOO}}(t)=0, \\ \frac{\max\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}}{1-\widehat{\pi}_i}, & Y_i(t)=0,\ \widehat{Y}_i^{\text{LOO}}(t)=1, \\ 0, & Y_i(t)=\widehat{Y}_i^{\text{LOO}}(t), \end{cases}$$

where $\widehat{\pi}_i \in (0,1)$ is the estimated semantic selectivity from O_i. The error weights mistakes on the minority class more heavily, such that propagation quality is assessed under class imbalance.

Propagation penalty. We add a confidence-based penalty $\rho^{(i)}_{\text{prop}}$ to account for statistical uncertainty from finite $|O_i|$ and the maximum scaled error under class imbalance:

$$\rho^{(i)}_{\text{prop}} = \Gamma^{(i)}_{\text{prop}} \sqrt{\frac{\ln(2|\mathcal{Q}^S|/\delta)}{2|O_i|}},$$

where $\delta \in (0,1)$ is a confidence parameter and $\Gamma^{(i)}_{\text{prop}} = \max\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}/\min\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}$ upper-bounds the scaled cost-sensitive error. The penalty increases when $|O_i|$ is small and shrinks as $|O_i|$ grows, yielding a conservative estimate of propagation quality.

Thus, the subjective score is defined as:

$$\widehat{\mathcal{L}}^{(i)}_{\text{subj}} = \widehat{\mathcal{L}}^{(i)}_{\text{LOO}} + \rho^{(i)}_{\text{prop}}.$$

Step 4: Rewrite construction. Given a trained tree-based model under candidate schema $\mathcal{R} \cup \mathcal{R}_k$, SEQR extracts positive decision paths, converts each path into a conjunctive predicate over attributes in $\mathcal{R} \cup \mathcal{R}_k$, ranks these predicates by support, and forms the rewritten query by taking the disjunction of the top-β_{rew} predicates. Here, the support of a positive path is the fraction of predicted positive tuples in $\Sigma_i(\mathcal{D}^E)$ that pass this path. Each selected path, therefore, contributes one conjunctive component, and the final rewrite is a union of the selected conjunctive predicates.

Step 5: Objective score computation. For each query q_i^S, SEQR evaluates the rewritten query against annotated or propagated labels in $\Sigma_i(\mathcal{D}^E)$. It first computes the empirical rewriting error and then adds a complexity-aware penalty.

Empirical rewriting error. Using a cost-sensitive rewriting error $\ell_{\text{rew}}(\cdot)$, the empirical rewriting error is defined as:

$$\widehat{\mathcal{L}}^{(i)}_{\text{rew}} = \frac{1}{|\Sigma_i(\mathcal{D}^E)|} \sum_{t \in \Sigma_i(\mathcal{D}^E)} \ell_{\text{rew}}\big(\widehat{Y}_i(t),\ q_i^T(t)\big),$$

where $\Sigma_i(\mathcal{D}^E)$ is the tuple subset satisfying the scope predicates of q_i^S under the enriched dataset $\mathcal{D}^E$, $\widehat{Y}_i(t) \in \{0,1\}$ is the propagated label associated with tuple t, and $q_i^T(t) \in \{0,1\}$ is the label predicted by executing q_i^T on $\mathcal{D}^E$.

Cost-sensitive rewriting error. Denote by the positive-label fraction in $\Sigma_i(\mathcal{D}^E)$ after label propagation as:

$$\widehat{\pi}_i^E = \frac{1}{|\Sigma_i(\mathcal{D}^E)|} \sum_{t \in \Sigma_i(\mathcal{D}^E)} \widehat{Y}_i(t)$$

We define the cost-sensitive rewriting error $\ell_{\text{rew}}(\cdot)$ as:

$$\ell_{\text{rew}}\big(\widehat{Y}_i(t), q_i^T(t)\big) = \begin{cases} \frac{\max\{\widehat{\pi}_i^E, 1-\widehat{\pi}_i^E\}}{\widehat{\pi}_i^E}, & \widehat{Y}_i(t)=1,\ q_i^T(t)=0, \\ \frac{\max\{\widehat{\pi}_i^E, 1-\widehat{\pi}_i^E\}}{1-\widehat{\pi}_i^E}, & \widehat{Y}_i(t)=0,\ q_i^T(t)=1, \\ 0, & \widehat{Y}_i(t)=q_i^T(t). \end{cases}$$

When $\widehat{\pi}_i^E$ is small or close to 1, the error amplifies mistakes on the rare class, so that rewrite fidelity is assessed under imbalance.

Rewrite penalty. We add a penalty to account for (a) finite-sample uncertainty in estimating $\widehat{\mathcal{L}}^{(i)}_{\text{rew}}$ from $|\Sigma_i(\mathcal{D}^E)|$ labeled tuples and (b) the bounded complexity of the rewritten query class, defined as:

$$\rho^{(i)}_{\text{rew}} = \Gamma^{(i)}_{\text{rew}} \sqrt{\frac{d^{(i)}_{\text{VC}} \ln(|\Sigma_i(\mathcal{D}^E)|) + \ln(2|\mathcal{Q}^S|/\delta)}{|\Sigma_i(\mathcal{D}^E)|}},$$

where $\delta \in (0,1)$ is the same confidence parameter as in $\rho^{(i)}_{\text{prop}}$, $\Gamma^{(i)}_{\text{rew}} = \frac{\max\{\widehat{\pi}_i^E, 1-\widehat{\pi}_i^E\}}{\min\{\widehat{\pi}_i^E, 1-\widehat{\pi}_i^E\}}$, and $d^{(i)}_{\text{VC}}$ is the VC dimension of the hypothesis class induced by q_i^T. Since each q_i^T is restricted to a union of at most β_{rew} conjunctive queries over the enriched schema $\mathcal{R}^E$, standard bounds [3, 6, 30] imply $d^{(i)}_{\text{VC}} = O\big(\beta_{\text{rew}} \cdot \text{arity}(\mathcal{R}^E) \cdot \log \beta_{\text{rew}}\big)$; we set *w.l.o.g.* $d^{(i)}_{\text{VC}} = \beta_{\text{rew}} \cdot \text{arity}(\mathcal{R}^E) \cdot \log \beta_{\text{rew}}$. The penalty grows with rewrite complexity and shrinks as $|\Sigma_i(\mathcal{D}^E)|$ increases, yielding a conservative estimate of rewrite fidelity.

Thus, the objective score is defined as:

$$\widehat{\mathcal{L}}^{(i)}_{\text{obj}} = \widehat{\mathcal{L}}^{(i)}_{\text{rew}} + \rho^{(i)}_{\text{rew}}.$$

Step 6: Candidate selection. For each candidate attribute set $\mathcal{R}_k$, SEQR combines their subjective and objective scores as

$$\mathcal{L}^{(i)}_{\text{static}} = \widehat{\mathcal{L}}^{(i)}_{\text{subj}} + \widehat{\mathcal{L}}^{(i)}_{\text{obj}},$$

and aggregates them across queries by

$$\frac{1}{|\mathcal{Q}^S|} \sum_{q_i^S \in \mathcal{Q}^S} \mathcal{L}^{(i)}_{\text{static}}.$$

It repeats Steps 2–5 for all candidate sets $\mathcal{R}_k$ with $k \in [1, \beta_{\text{SE}}]$, and returns the candidate set whose aggregated score is minimum as the final enriched schema configuration.

C PROOF OF LEMMA 2

Lemma 2: *Let O_i be drawn* i.i.d. *from $\Sigma_i(\mathcal{D})$ and let $\mathcal{L}^{(i)}_{\text{prop}}$ denote the expected propagation error on an independently drawn tuple from $\Sigma_i(\mathcal{D})$. For any deterministic label propagation algorithm,*

$$\mathbb{E}\Big[\widehat{\mathcal{L}}^{(i)}_{\text{LOO}}\Big] = \mathcal{L}^{(i)}_{\text{prop}},$$

where the expectation $\mathbb{E}$ is over O_i and the held-out tuple. □

Proof: For a uniformly chosen tuple $t \in O_i$, denote by $Y_i(t) \in \{0,1\}$ the ground-truth label of t, $\widehat{Y}_i^{\text{LOO}}(t)$ the propagated label of t produced by the deterministic label propagation algorithm based on the labeled set $O_i \setminus \{t\}$, and

$$\Delta(t) = \ell^{(i)}_{\text{prop}}\big(Y_i(t),\ \widehat{Y}_i^{\text{LOO}}(t)\big),$$

the leave-one-out propagation error on t.

For an independently drawn tuple $t' \in \Sigma_i(\mathcal{D})$, denote by $Y_i(t')$ the ground-truth label of t', and $\widehat{Y}_i(t'; O_i)$ the propagated label on t' when the propagation algorithm is run with labeled input O_i.

Denote by $\mathcal{L}^{(i)}_{\text{prop}}$ the expected propagation error on t',

$$\mathcal{L}^{(i)}_{\text{prop}} = \mathbb{E}_{t' \in \Sigma_i(\mathcal{D})}\Big[\ell^{(i)}_{\text{prop}}\big(Y_i(t'),\ \widehat{Y}_i(t'; O_i)\big)\Big].$$

Since O_i is drawn *i.i.d.* from $\Sigma_i(\mathcal{D})$ and t is selected uniformly from O_i, the joint distribution of $\big(Y_i(t),\ O_i \setminus \{t\}\big)$ coincides with that of $\big(Y_i(t'),\ O_i\big)$ by the standard exchangeability of *i.i.d.* samples, which underlies leave-one-out and cross-validation analyses [7, 51].

Applying the same deterministic label propagation algorithm (a

measurable function of its labeled input) to $O_i \setminus \{t\}$ or O_i preserves this distributional equivalence. Thus, the leave-one-out prediction $\widehat{Y}_i^{\mathrm{LOO}}(t)$ has the same distribution as $\widehat{Y}_i(t'; O_i)$, and consequently,

$$\mathbb{E}[\Delta(t)] = \mathbb{E}\Big[\ell^{(i)}_{\mathrm{prop}}\big(Y_i(t'),\ \widehat{Y}_i(t'; O_i)\big)\Big] = \mathcal{L}^{(i)}_{\mathrm{prop}}.$$

By the definition of $\widehat{\mathcal{L}}^{(i)}_{\mathrm{LOO}}$, linearity of expectation, and exchangeability of the tuples in O_i, we conclude that

$$\mathbb{E}\Big[\widehat{\mathcal{L}}^{(i)}_{\mathrm{LOO}}\Big] = \mathbb{E}\left[\frac{1}{|O_i|}\sum_{t\in O_i}\Delta(t)\right] = \frac{1}{|O_i|}\sum_{t\in O_i}\mathbb{E}[\Delta(t)] = \mathcal{L}^{(i)}_{\mathrm{prop}}. \qquad \square$$

D PROOF OF THEOREM 3

Theorem 3: *Given a semantic query set $\mathcal{Q}^S$, enriched data $\mathcal{D}^E$ under schema $\mathcal{R}^E$ (derived from $\mathcal{D}$ and $\mathcal{U}$), and the results $\mathcal{E}$ of executing the rewritten* SQL *workload $\mathcal{Q}^T$ produced by* SEQR*, for any $\delta \in (0,1)$ the following bound holds with probability at least $1-\delta$:*

$$\mathrm{acc}(\mathcal{Q}^S, \mathcal{E}) \ \geq\ \frac{1}{|\mathcal{Q}^S|}\sum_{q_i^S\in\mathcal{Q}^S}\Phi(\pi_i, \mathcal{L}^{(i)}_{\mathrm{opt}}),$$

$$\Phi(\pi_i, \mathcal{L}^{(i)}_{\mathrm{opt}}) = \begin{cases} \dfrac{2(\pi_i - \mathcal{L}^{(i)}_{\mathrm{opt}})}{2\pi_i - \mathcal{L}^{(i)}_{\mathrm{opt}}}, & 0 \leq \mathcal{L}^{(i)}_{\mathrm{opt}} < \pi_i, \\ 0, & \mathcal{L}^{(i)}_{\mathrm{opt}} \geq \pi_i, \end{cases}$$

where $\pi_i = \frac{|G_i|}{|\Sigma_i(\mathcal{D})|}$ is the true selectivity of q_i^S on $\Sigma_i(\mathcal{D})$, $\mathcal{L}^{(i)}_{\mathrm{opt}}$ is the aggregated error of q_i^S that yields the minimum averaged score, and $\Phi(\cdot)$ maps $(\pi_i, \mathcal{L}^{(i)}_{\mathrm{opt}})$ to a lower bound on the F1 score of q_i^S. $\square$

Proof: We prove this by first deriving a lower bound on the true F_1 score of a fixed rewritten query q_i^T of $q_i^S \in \mathcal{Q}^S$, and then extending the result to the macro-average over all queries in $\mathcal{Q}^S$.

(S1) Upper bounding the true cost-sensitive error. By definition, the true cost-sensitive error of q_i^T is

$$\mathcal{L}^{(i)}_{\mathrm{true}} = \mathbb{E}_{t\in\Sigma_i(\mathcal{D}^E)}\Big[\ell^{(i)}_{\mathrm{prop}}\big(Y_i(t), \widehat{Y}_i(t)\big) + \ell^{(i)}_{\mathrm{rew}}\big(\widehat{Y}_i(t), q_i^T(t)\big)\Big]$$
$$= \underbrace{\mathbb{E}_{t\in\Sigma_i(\mathcal{D}^E)}\big[\ell^{(i)}_{\mathrm{prop}}(Y_i(t), \widehat{Y}_i(t))\big]}_{\mathcal{L}^{(i)}_{\mathrm{prop}}} + \underbrace{\mathbb{E}_{t\in\Sigma_i(\mathcal{D}^E)}\big[\ell^{(i)}_{\mathrm{rew}}(\widehat{Y}_i(t), q_i^T(t))\big]}_{\mathcal{L}^{(i)}_{\mathrm{rew}}}.$$

Here $\ell^{(i)}_{\mathrm{prop}}(Y_i(t), \widehat{Y}_i(t))$ measures the discrepancy between the propagated label and the true label, while $\ell^{(i)}_{\mathrm{rew}}(\widehat{Y}_i(t), q_i^T(t))$ measures the discrepancy between the rewritten query output and the propagated label. Thus $\mathcal{L}^{(i)}_{\mathrm{prop}}$ and $\mathcal{L}^{(i)}_{\mathrm{rew}}$ are the true propagation and rewriting errors, respectively.

Since LiteDBX observes ground-truth labels only on the subset O_i, it cannot directly compute the true propagation error $\mathcal{L}^{(i)}_{\mathrm{prop}}$ on $\Sigma_i(\mathcal{D}^E)$. Instead, it constructs a high-probability upper bound via the LOO estimator on O_i, as defined in Section 5:

$$\mathcal{L}^{(i)}_{\mathrm{prop}} \ \leq\ \mathcal{L}^{(i)}_{\mathrm{subj}} = \widehat{\mathcal{L}}^{(i)}_{\mathrm{LOO}} + \Gamma^{(i)}_{\mathrm{prop}}\sqrt{\frac{\ln\big(2|\mathcal{Q}^S|/\delta\big)}{2|O_i|}},$$

where the bound quantifies propagation error *w.r.t.* the (unknown) ground truth. Moreover, since the rewriting loss compares the rewritten query output to the propagated labels $\widehat{Y}_i$ on $\Sigma_i(\mathcal{D}^E)$, LiteDBX can compute an empirical rewriting error over $\Sigma_i(\mathcal{D}^E)$ and upper-bound its expected value via VC theory (Section 5):

$$\mathcal{L}^{(i)}_{\mathrm{rew}} \leq \mathcal{L}^{(i)}_{\mathrm{obj}} = \widehat{\mathcal{L}}^{(i)}_{\mathrm{rew}} + \Gamma^{(i)}_{\mathrm{rew}}\sqrt{\frac{d^{(i)}_{\mathrm{VC}}\ \ln(|\Sigma_i(\mathcal{D}^E)|) + \ln\big(2|\mathcal{Q}^S|/\delta\big)}{|\Sigma_i(\mathcal{D}^E)|}}.$$

We next show that these inequalities hold with probability at least $1 - \frac{\delta}{2|\mathcal{Q}^S|}$, giving a high-probability upper bound on $\mathcal{L}^{(i)}_{\mathrm{true}}$.

Bounding true propagation error. By Lemma 2, $\widehat{\mathcal{L}}^{(i)}_{\mathrm{LOO}}$ is an unbiased estimator of $\mathcal{L}^{(i)}_{\mathrm{prop}}$. Moreover, for each held-out tuple $t \in O_i$, the leave-one-out loss $\ell^{(i)}_{\mathrm{prop}}\big(Y_i(t),\ \widehat{Y}_i^{\mathrm{LOO}}(t)\big)$ is bounded in $[0, \Gamma^{(i)}_{\mathrm{prop}}]$, where $\Gamma^{(i)}_{\mathrm{prop}} = \frac{\max\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}}{\min\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}}$ and $\widehat{\pi}_i = \frac{1}{|O_i|}\sum_{t\in O_i} Y_i(t)$.

Since O_i is drawn *i.i.d.* from $\Sigma_i(\mathcal{D})$ and the LOO estimator is a bounded symmetric statistic, Hoeffding-type concentration for leave-one-out averages [25] implies that for any $\varepsilon > 0$,

$$\Pr\Big(\widehat{\mathcal{L}}^{(i)}_{\mathrm{LOO}} - \mathcal{L}^{(i)}_{\mathrm{prop}} \geq \varepsilon\Big) \leq \exp\left(\frac{-2|O_i|\varepsilon^2}{{\Gamma^{(i)}_{\mathrm{prop}}}^2}\right).$$

Equivalently,

$$\Pr\Big(\mathcal{L}^{(i)}_{\mathrm{prop}} \leq \widehat{\mathcal{L}}^{(i)}_{\mathrm{LOO}} + \varepsilon\Big) \geq 1 - \exp\left(\frac{-2|O_i|\varepsilon^2}{{\Gamma^{(i)}_{\mathrm{prop}}}^2}\right).$$

Given a target confidence parameter $\delta \in (0,1)$, letting

$$\varepsilon = \Gamma^{(i)}_{\mathrm{prop}}\sqrt{\frac{\ln\big(2|\mathcal{Q}^S|/\delta\big)}{2|O_i|}}$$

yields

$$\Pr\left(\mathcal{L}^{(i)}_{\mathrm{prop}} \leq \widehat{\mathcal{L}}^{(i)}_{\mathrm{LOO}} + \Gamma^{(i)}_{\mathrm{prop}}\sqrt{\frac{\ln\big(2|\mathcal{Q}^S|/\delta\big)}{2|O_i|}}\right) \geq 1 - \frac{\delta}{2|\mathcal{Q}^S|}.$$

Bounding true rewriting error. To upper-bound the true rewriting error, we apply a uniform generalization bound from VC theory [57] to the empirical rewriting error

$$\widehat{\mathcal{L}}^{(i)}_{\mathrm{rew}} = \frac{1}{|\Sigma_i(\mathcal{D}^E)|}\sum_{t\in\Sigma_i(\mathcal{D}^E)}\ell^{(i)}_{\mathrm{rew}}\big(\widehat{Y}_i(t), q_i^T(t)\big).$$

Each rewritten query q_i^T is restricted to a union of at most β_{rew} conjunctive queries over the enriched schema $\mathcal{R}^E$. Accordingly, let $\mathcal{H}^{(i)}$ denote the hypothesis class of such UCQs, each mapping tuples in $\Sigma_i(\mathcal{D}^E)$ to $\{0,1\}$ and thus acting as a Boolean classifier.

Denote by $d^{(i)}_{\mathrm{VC}}$ the VC dimension of $\mathcal{H}^{(i)}$. Since a UCQ corresponds to the disjunction of β_{rew} conjunctions over $\mathrm{arity}(\mathcal{R}^E)$ predicate classes, standard results [3, 6, 30] on the VC dimension of finite unions imply

$$d^{(i)}_{\mathrm{VC}} = \beta_{\mathrm{rew}} \cdot \mathrm{arity}(\mathcal{R}^E) \cdot \log \beta_{\mathrm{rew}}.$$

Since the cost-sensitive rewriting error is bounded in $[0, \Gamma^{(i)}_{\mathrm{rew}}]$, where $\Gamma^{(i)}_{\mathrm{rew}} = \max\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}/\min\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}$ and $\widehat{\pi}_i = \frac{1}{|\Sigma_i(\mathcal{D}^E)|}\sum_{t\in\Sigma_i(\mathcal{D}^E)}\widehat{Y}_i(t)$, the VC inequality implies that for any $\delta \in (0,1)$, with probability at least $1 - \frac{\delta}{2|\mathcal{Q}^S|}$,

$$\mathcal{L}^{(i)}_{\mathrm{rew}} \ \leq\ \widehat{\mathcal{L}}^{(i)}_{\mathrm{rew}} \ +\ \Gamma^{(i)}_{\mathrm{rew}}\sqrt{\frac{d^{(i)}_{\mathrm{VC}}\ln|\Sigma_i(\mathcal{D}^E)| + \ln\big(2|\mathcal{Q}^S|/\delta\big)}{|\Sigma_i(\mathcal{D}^E)|}}.$$

Union bound of true propagation and rewriting error. Each of the bounds for $\mathcal{L}^{(i)}_{\mathrm{prop}}$ and $\mathcal{L}^{(i)}_{\mathrm{rew}}$ holds with probability at least $1 - \frac{\delta}{2|\mathcal{Q}^S|}$.

By the union bound, both inequalities hold simultaneously with probability at least $1 - \frac{\delta}{|Q^S|}$, and therefore

$$\begin{aligned}\mathcal{L}^{(i)}_{\text{true}} &= \mathcal{L}^{(i)}_{\text{prop}} + \mathcal{L}^{(i)}_{\text{rew}} \\ &\le \widehat{\mathcal{L}}^{(i)}_{\text{LOO}} + \widehat{\mathcal{L}}^{(i)}_{\text{rew}} + \Gamma^{(i)}_{\text{prop}} \sqrt{\frac{\ln(2|Q^S|/\delta)}{2|O_i|}} \\ &\quad + \Gamma^{(i)}_{\text{rew}} \sqrt{\frac{d^{(i)}_{\text{VC}} \ln|\Sigma_i(\mathcal{D}^E)| + \ln(2|Q^S|/\delta)}{|\Sigma_i(\mathcal{D}^E)|}} \\ &= \mathcal{L}^{(i)}_{\text{opt}},\end{aligned}$$

which completes the upper bound on $\mathcal{L}^{(i)}_{\text{true}}$.

(S2) Bounding the true 0–1 error via cost-sensitive error. The cost-sensitive errors $\ell^{(i)}_{\text{prop}}$ and $\ell^{(i)}_{\text{rew}}$ are instantiated using an empirical positive rate $\widehat{\pi}_i$ as defined in Section 5 (for propagation on O_i and, by overloading, for rewriting on $\Sigma_i(\mathcal{D}^E)$). In both cases, since $0 < \widehat{\pi}_i < 1$, any false negative or false positive incurs cost at least 1.

Thus, for every tuple $t \in \Sigma_i(\mathcal{D}^E)$,

$$\mathbb{1}[q^T_i(t) \neq Y_i(t)] \;\le\; \ell^{(i)}_{\text{prop}}(Y_i(t), \widehat{Y}_i(t)) + \ell^{(i)}_{\text{rew}}(\widehat{Y}_i(t), q^T_i(t)),$$

because any error in label propagation or query rewriting must incur at least unit cost under the above penalty scheme.

Denote by $\text{Err}^{(i)}$, the true 0–1 error of q^T_i. Taking expectations over $t \in \Sigma_i(\mathcal{D}^E)$ gives

$$\text{Err}^{(i)} = \Pr_{t \in \Sigma_i(\mathcal{D}^E)}\left[q^T_i(t) \neq Y_i(t)\right] \;\le\; \mathcal{L}^{(i)}_{\text{prop}} + \mathcal{L}^{(i)}_{\text{rew}} = \mathcal{L}^{(i)}_{\text{true}}.$$

From (S1), with probability at least $1 - \frac{\delta}{|Q^S|}$, $\mathcal{L}^{(i)}_{\text{true}} \le \mathcal{L}^{(i)}_{\text{opt}}$. Hence, with probability at least $1 - \frac{\delta}{|Q^S|}$:

$$\text{Err}^{(i)} \;\le\; \mathcal{L}^{(i)}_{\text{opt}}.$$

(S3) Lower bounding the F_1 score via the true 0-1 error. Denote by TP_i, FP_i, FN_i and TN_i the probabilities of true positive, false positive, false negative and true negative events of q^T_i on $\Sigma_i(\mathcal{D}^E)$, respectively.

$$\begin{aligned}\text{TP}_i &= \Pr_{t \in \Sigma_i(\mathcal{D}^E)}\big(Y_i(t) = 1,\; q^T_i(t) = 1\big), \\ \text{FP}_i &= \Pr_{t \in \Sigma_i(\mathcal{D}^E)}\big(Y_i(t) = 0,\; q^T_i(t) = 1\big), \\ \text{FN}_i &= \Pr_{t \in \Sigma_i(\mathcal{D}^E)}\big(Y_i(t) = 1,\; q^T_i(t) = 0\big), \\ \text{TN}_i &= \Pr_{t \in \Sigma_i(\mathcal{D}^E)}\big(Y_i(t) = 0,\; q^T_i(t) = 0\big).\end{aligned}$$

By definition of selectivity and 0–1 error,

$$\pi_i = \Pr_{t \in \Sigma_i(\mathcal{D}^E)}\left[Y_i(t) = 1\right] = \text{TP}_i + \text{FN}_i,$$

where $\Sigma_i(\mathcal{D}^E)$ and $\Sigma_i(\mathcal{D})$ contain the same tuple ids since enrichment only adds attributes.

$$\text{Err}^{(i)} = \Pr_{t \in \Sigma_i(\mathcal{D}^E)}\left[q^T_i(t) \neq Y_i(t)\right] = \text{FP}_i + \text{FN}_i.$$

Consequently, we obtain the feasible range of FN_i as follows:

$$\begin{aligned}\text{TP}_i = \pi_i - \text{FN}_i \;\ge\; 0 \quad &\Longrightarrow \quad \text{FN}_i \;\le\; \pi_i, \\ \text{FP}_i = \text{Err}^{(i)} - \text{FN}_i \;\ge\; 0 \quad &\Longrightarrow \quad \text{FN}_i \;\le\; \text{Err}^{(i)}.\end{aligned}$$

Moreover, $\text{TN}_i = 1 - \text{TP}_i - \text{FP}_i - \text{FN}_i = 1 - \pi_i - \big(\text{Err}^{(i)} - \text{FN}_i\big) \;\ge\; 0$. It implies $\text{FN}_i \;\ge\; \text{Err}^{(i)} - (1 - \pi_i)$. As $\text{FN}_i \ge 0$, we obtain

$$\text{FN}_i \in \Big[\max\{0, \text{Err}^{(i)} - (1 - \pi_i)\},\; \min\{\text{Err}^{(i)}, \pi_i\}\Big].$$

The F_1 score of q^T_i is

$$F_1(q^T_i) = \frac{2\text{TP}_i}{2\text{TP}_i + \text{FP}_i + \text{FN}_i}.$$

Using $\text{TP}_i = \pi_i - \text{FN}_i$ and $\text{FP}_i = \text{Err}^{(i)} - \text{FN}_i$, we rewrite

$$\begin{aligned}F_1(q^T_i) &= \frac{2(\pi_i - \text{FN}_i)}{2(\pi_i - \text{FN}_i) + \text{FP}_i + \text{FN}_i} \\ &= \frac{2(\pi_i - \text{FN}_i)}{2(\pi_i - \text{FN}_i) + \text{Err}^{(i)}}.\end{aligned}$$

Given the fixed π_i and $\text{Err}^{(i)}$, we treat $F_1(q^T_i)$ as a function of FN_i on its feasible interval, and compute its derivative as

$$\begin{aligned}&\frac{\partial}{\partial \text{FN}_i}\left(\frac{2(\pi_i - \text{FN}_i)}{2(\pi_i - \text{FN}_i) + \text{Err}^{(i)}}\right) \\ &= \frac{-2\big(2(\pi_i - \text{FN}_i) + \text{Err}^{(i)}\big) - 2(\pi_i - \text{FN}_i)\cdot(-2)}{\big(2(\pi_i - \text{FN}_i) + \text{Err}^{(i)}\big)^2} \\ &= \frac{-2\,\text{Err}^{(i)}}{\big(2(\pi_i - \text{FN}_i) + \text{Err}^{(i)}\big)^2} \;\le\; 0.\end{aligned}$$

Thus, for fixed π_i and $\text{Err}^{(i)}$, $F_1(q^T_i)$ is nonincreasing in FN_i on its feasible interval. Thus the lower bound of $F_1(q^T_i)$ occurs at the largest feasible FN_i.

We consider two cases.

- *Case 1:* $\text{Err}^{(i)} \ge \pi_i$. We have $\min\{\text{Err}^{(i)}, \pi_i\} = \pi_i$, and the largest feasible FN_i is π_i, which forces $\text{TP}_i = 0$. Hence $F_1(q^T_i) = 0$.
- *Case 2:* $\text{Err}^{(i)} < \pi_i$. We have $\min\{\text{Err}^{(i)}, \pi_i\} = \text{Err}^{(i)}$, and the largest feasible FN_i is $\text{Err}^{(i)}$. This yields $\text{FP}_i = 0$ and $\text{TP}_i = \pi_i - \text{Err}^{(i)}$, such that

$$F_1(q^T_i) \;\ge\; \frac{2(\pi_i - \text{Err}^{(i)})}{2(\pi_i - \text{Err}^{(i)}) + \text{Err}^{(i)}} = \frac{2(\pi_i - \text{Err}^{(i)})}{2\pi_i - \text{Err}^{(i)}}.$$

Combining the two cases, we obtain a deterministic lower bound in terms of the selectivity π_i and the true error $\text{Err}^{(i)}$:

$$F_1(q^T_i) \;\ge\; \Phi\big(\pi_i,\, \text{Err}^{(i)}\big),$$

where

$$\Phi(\pi_i,\, \text{Err}^{(i)}) = \begin{cases}\dfrac{2(\pi_i - \text{Err}^{(i)})}{2\pi_i - \text{Err}^{(i)}}, & 0 \le \text{Err}^{(i)} < \pi_i, \\ 0, & \text{Err}^{(i)} \ge \pi_i.\end{cases}$$

Moreover, when $\pi_i \in (0, 1)$ is fixed and $\text{Err}^{(i)}$ is a variable, the function $\Phi(\pi_i, \text{Err}^{(i)})$ is nonincreasing on $[0, 1]$, since its derivative with respect to $\text{Err}^{(i)}$ is

$$\frac{\partial}{\partial \text{Err}^{(i)}}\left(\frac{2(\pi_i - \text{Err}^{(i)})}{2\pi_i - \text{Err}^{(i)}}\right) = \frac{-2\pi_i}{(2\pi_i - \text{Err}^{(i)})^2} \;\le\; 0.$$

From (S2), with probability at least $1 - \frac{\delta}{|Q^S|}$ we have $\text{Err}^{(i)} \le \mathcal{L}^{(i)}_{\text{opt}}$. By the monotonicity of $\Phi(\pi_i, \cdot)$ in its second argument, this implies that, with probability at least $1 - \frac{\delta}{|Q^S|}$,

$$F_1(q^T_i) \;\ge\; \Phi\big(\pi_i, \text{Err}^{(i)}\big) \;\ge\; \Phi\big(\pi_i, \mathcal{L}^{(i)}_{\text{opt}}\big).$$

(S4) Extension to the macro-averaged F_1 score.

By the definition of macro-averaged F_1,

$$\mathrm{acc}(Q^S, \mathcal{E}, \mathcal{D}, \mathcal{U}) = \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} F_1(q_i^T).$$

Applying a union bound to the per-query event in (S3), we apply a union bound over each query $q_i^S \in Q^S$ and conclude that with probability at least $1-\delta$,

$$\mathrm{acc}(Q^S, \mathcal{E}, \mathcal{D}, \mathcal{U}) \;\geq\; \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} \Phi\big(\pi_i, \mathcal{L}_{\mathrm{opt}}^{(i)}\big),$$

which completes the proof of Theorem 3. □

E DETAILS FOR CERTIFIED REUSE

This section gives additional details for certified reuse in Section 6.

Step 1: Updating enriched data. Given the enriched schema $\mathcal{R}^E$ and data $\mathcal{D}^E$ produced by SEQR, it first applies incoming updates to the underlying data, including tuple insertions, deletions, and value updates (treated as delete–insert operations). It then applies the query-scope predicates Σ_i of each query $q_i^S \in Q^S$ to the updated data, materializes the required attributes in $\mathcal{R}^E$ for the resulting newly introduced tuples using the same feature-population procedure as in Section B, and obtains the updated enriched data $\mathcal{D}^E_{\mathrm{new}}$.

Step 2: Updating coresets. For each query q_i^S, LiteDBX constructs the updated coreset $C_{\mathrm{new}}^{(i)}$ by augmenting the original coreset C_i with newly introduced tuples. For each tuple $t \in \Sigma_i(\mathcal{D}^E_{\mathrm{new}}) \setminus \Sigma_i(\mathcal{D}^E)$, it computes the selectivity-aware score $s(t)$ using the same confidence construction as in Section B. Denote by τ_i^+ (resp. τ_i^-) the minimum (resp. maximum) selectivity-aware score among tuples in the original coreset C_i labeled as positive (resp. negative). A newly introduced tuple t is added to $C_{\mathrm{new}}^{(i)}$ if

$$s(t) > \tau_i^+ \quad \text{or} \quad s(t) < \tau_i^-.$$

Thus, $C_{\mathrm{new}}^{(i)}$ preserves the high-confidence thresholds induced by C_i while incorporating newly informative tuples under the updated data distribution.

Step 3: Re-propagating labels. Using the updated coreset $C_{\mathrm{new}}^{(i)}$, it reruns the same label-propagation procedure as in Section B under the fixed schema $\mathcal{R}^E$. Specifically, it trains the same tree-based model on $C_{\mathrm{new}}^{(i)}$ and applies the trained model to tuples in $\Sigma_i(\mathcal{D}^E_{\mathrm{new}}) \setminus C_{\mathrm{new}}^{(i)}$ to obtain updated propagated labels, denoted by $\widehat{Y}_i^{\mathrm{new}}(t)$.

Step 4: Computing the incremental error certificate. Given the original propagated labels $\widehat{Y}_i(\cdot)$ on $\mathcal{D}^E$ and the updated propagated labels $\widehat{Y}_i^{\mathrm{new}}(\cdot)$ on $\mathcal{D}^E_{\mathrm{new}}$, LiteDBX computes the incremental error certificate for each query q_i^S as

$$\Delta^{(i)} = \frac{|\Sigma_i(\mathcal{D}^E_{\mathrm{new}}) \setminus \Sigma_i(\mathcal{D}^E)|}{|\Sigma_i(\mathcal{D}^E_{\mathrm{new}})|} + \frac{\sum_{t \in \Sigma_i(\mathcal{D}^E) \cap \Sigma_i(\mathcal{D}^E_{\mathrm{new}})} \mathbb{1}\big[\widehat{Y}_i(t) \neq \widehat{Y}_i^{\mathrm{new}}(t)\big]}{|\Sigma_i(\mathcal{D}^E) \cap \Sigma_i(\mathcal{D}^E_{\mathrm{new}})|}.$$

The first term measures the update mass contributed by newly introduced tuples, while the second measures propagation instability on previously covered tuples after the coreset update. Together, they quantify the extent to which the updated data may invalidate the current answer configuration.

Step 5: Reuse test. Denote by $\eta > 0$, a user-specified tolerance on the certified error increase. For each query q_i^S, LiteDBX reuses the current rewrites only if

$$\big(\Gamma_{\mathrm{prop}}^{(i)} + \Gamma_{\mathrm{rew}}^{(i)}\big)\Delta^{(i)} \leq \eta;$$

otherwise, it triggers selective refresh.

F PROOF OF THEOREM 4

Theorem 4: *Given $\mathcal{R}^E$, $\mathcal{D}^E$, Q^T, $\widehat{\pi}_i$, $\widehat{\pi}_i^E$ and $\mathcal{L}_{\mathrm{opt}}^{(i)}$ for each query q_i^S produced by* SEQR *under confidence level δ, denote by $\mathcal{L}_{\mathrm{update}}^{(i)}$ the aggregated error of q_i^S on the updated dataset $\mathcal{D}^E_{\mathrm{new}}$ when evaluated under the same schema $\mathcal{R}^E$ and rewritten queries Q^T. Then, with probability at least $1-\delta$, the following bound holds:*

$$\mathcal{L}_{\mathrm{update}}^{(i)} \;\leq\; \mathcal{L}_{\mathrm{opt}}^{(i)} \;+\; \big(\Gamma_{\mathrm{prop}}^{(i)} + \Gamma_{\mathrm{rew}}^{(i)}\big) \cdot \Delta^{(i)}.$$

Here $\Gamma_{\mathrm{prop}}^{(i)} = \frac{\max\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}}{\min\{\widehat{\pi}_i, 1-\widehat{\pi}_i\}}$ and $\Gamma_{\mathrm{rew}}^{(i)} = \frac{\max\{\widehat{\pi}_i^E, 1-\widehat{\pi}_i^E\}}{\min\{\widehat{\pi}_i^E, 1-\widehat{\pi}_i^E\}}$, where $\widehat{\pi}_i$ is the selectivity estimated on the observed set O_i, and $\widehat{\pi}_i^E$ is the fraction of tuples with positive propagated labels in $\Sigma_i(\mathcal{D}^E)$. □

Proof: Under the fixed enriched schema $\mathcal{R}^E$ and the same supervision O_i, any change in the propagation error between $\mathcal{D}^E$ and $\mathcal{D}^E_{\mathrm{new}}$ can only arise from differences in propagated labels. We prove this in two steps by decomposing the propagation error accordingly.

For brevity, in this proof we overload $\mathcal{D}^E$ (resp. $\mathcal{D}^E_{\mathrm{new}}$) to denote the SQL-filtered tuple set $\Sigma_i(\mathcal{D}^E)$ (resp. $\Sigma_i(\mathcal{D}^E_{\mathrm{new}})$) for query q_i^S.

Recall that the aggregated error score in SEQR is the sum of a propagation component and a rewriting component. In the dynamic setting, we keep the rewritten query $q_i^T \in Q^T$ fixed. However, as the rewriting error is evaluated against propagated labels, it may still change due to changes in propagated labels. Therefore, it suffices to upper-bound the increases of both the propagation and rewriting components under data updates.

(S1) Bounding propagation error on newly added tuples. Denote by $\mathcal{D}^+ = \mathcal{D}^E_{\mathrm{new}} \setminus \mathcal{D}^E$, the set of newly added tuples where value updates are modeled as delete–insert operations.

By definition, the true propagation error of query q_i^S on $\mathcal{D}^E$ is

$$\mathcal{L}_{\mathrm{prop}}^{(i)} = \frac{1}{|\mathcal{D}^E|} \sum_{t \in \mathcal{D}^E} \ell_{\mathrm{prop}}\big(Y_i(t), \widehat{Y}_i(t)\big),$$

and the corresponding propagation error on $\mathcal{D}^E_{\mathrm{new}}$ is

$$\mathcal{L}_{\mathrm{prop,update}}^{(i)} = \frac{1}{|\mathcal{D}^E_{\mathrm{new}}|} \sum_{t \in \mathcal{D}^E_{\mathrm{new}}} \ell_{\mathrm{prop}}\big(Y_i(t), \widehat{Y}_i^{\mathrm{new}}(t)\big),$$

where $\widehat{Y}_i(t)$ and $\widehat{Y}_i^{\mathrm{new}}(t)$ are the propagated labels produced on $\mathcal{D}^E$ and $\mathcal{D}^E_{\mathrm{new}}$, respectively, by running the same deterministic propagation procedure with the fixed ground truth labels O_i.

For any $t \in \mathcal{D}^+$, there is no corresponding propagated label under $\mathcal{D}^E$. By the boundedness of the cost-sensitive error,

$$\ell_{\mathrm{prop}}\big(Y_i(t), \widehat{Y}_i^{\mathrm{new}}(t)\big) \;\leq\; \Gamma_{\mathrm{prop}}^{(i)},$$

where $\Gamma_{\mathrm{prop}}^{(i)}$ is the maximum possible propagation error under the scaled cost-sensitive error.

Therefore, the contribution of newly added tuples to the propagation error on $\mathcal{D}^E_{\mathrm{new}}$ is bounded by

$$\frac{1}{|\mathcal{D}^E_{\mathrm{new}}|} \sum_{t \in \mathcal{D}^+} \ell_{\mathrm{prop}}\big(Y_i(t), \widehat{Y}_i^{\mathrm{new}}(t)\big) \;\leq\; \Gamma_{\mathrm{prop}}^{(i)} \cdot \frac{|\mathcal{D}^E_{\mathrm{new}} \setminus \mathcal{D}^E|}{|\mathcal{D}^E_{\mathrm{new}}|}.$$

(S2) Bounding propagation error on shared tuples. We next

Datasets	Q^S	LiteDBX			LOTUS			Palimpzest			ThalamusDB			BigQuery			LOTUS+Cache		
		F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)
Movie	Q_1	**0.97**	**0.0003**	**6.51**	0.88	0.8520	992.96	0.88	0.8600	939.53	0.46	0.6800	2262.36	0.85	0.1037	121.24	0.86	**0.0003**	6.78
	Q_2	**1.00**	**0.0002**	**3.59**	0.80	0.0490	67.00	0.80	0.0380	89.68	0.78	0.0780	263.27	0.60	0.0062	8.48	0.67	**0.0002**	3.88
Wildlife	Q_1	**1.00**	**0.0005**	**9.20**	0.89	0.1100	406.98	**1.00**	0.0660	146.10	0.33	0.1000	251.90	**1.00**	0.2200	24.60	**1.00**	**0.0005**	9.22
E-Comm.	Q_1	**1.00**	**0.0002**	**4.59**	**1.00**	0.9800	881.00	**1.00**	0.8300	711.23	0.73	1.3700	1115.02	**1.00**	0.2142	221.53	0.92	**0.0002**	5.70
	Q_2	**1.00**	**0.0003**	**8.60**	0.67	1.1500	1996.78	0.50	1.2700	418.23	0.22	0.2880	224.67	0.30	1.5122	1808.68	0.73	**0.0003**	8.85
	Q_3	**0.80**	**0.0006**	**6.40**	0.64	1.2900	1978.40	0.51	1.5200	890.00	✗	✗	✗	0.58	0.7551	276.19	0.64	**0.0006**	7.51
MMQA	Q_1	0.83	**0.0004**	7.44	0.76	0.0750	111.61	0.76	0.0426	51.00	0.80	0.1000	503.26	0.80	0.0151	**2.46**	**0.85**	**0.0004**	7.46
	Q_2	**1.00**	**0.0004**	7.90	0.75	0.0800	77.07	0.85	0.0430	33.22	**1.00**	0.9700	370.85	0.60	0.0152	**4.14**	0.86	**0.0004**	7.92
	Q_3	**1.00**	**0.0003**	**0.68**	0.28	0.0771	138.41	**1.00**	0.0322	54.14	0.09	0.1500	1332.12	0.03	0.0096	18.90	0.22	**0.0003**	6.09
	Q_4	**1.00**	**0.0004**	**7.76**	0.20	0.0900	104.59	**1.00**	0.0320	25.11	0.08	0.2400	1237.96	0.04	0.0095	17.30	0.29	**0.0004**	7.78
	Q_5	**1.00**	**0.0002**	**4.96**	0.39	0.1000	128.46	**1.00**	0.0330	52.36	0.07	0.2400	1062.16	0.13	0.0097	17.40	0.43	**0.0002**	4.99
Medical	Q_1	**0.52**	**0.0005**	**8.31**	0.33	0.5800	683.00	0.26	0.7000	689.12	0.32	0.8200	2350.88	0.19	0.0465	59.01	0.32	**0.0005**	9.31
	Q_2	0.84	**0.0004**	**7.39**	0.79	0.1670	307.41	**0.88**	0.2680	271.33	0.19	0.0730	292.17	0.85	2.7591	1103.59	0.79	**0.0004**	8.41
	Q_3	**0.63**	**0.0003**	**4.95**	0.36	0.0520	77.50	0.61	0.1023	115.85	0.28	0.0360	311.81	**0.63**	0.3598	484.13	0.35	**0.0003**	5.19
	Q_4	0.51	**0.0004**	**4.50**	0	0.1800	291.63	0.31	0.2800	176.28	TO	TO	TO	**0.56**	0.2900	98.14	0	**0.0004**	4.79

Table 6: Comparison of LiteDBX and baselines on static semantic query workloads. For each dataset, we report F_1, monetary cost, and runtime for each query. Bold (resp. underlined) entries indicate the best (resp. second-best) results; ✗ (resp. TO) denotes unsupported workload (resp. timeout after 1 hour).

bound the propagation error change on tuples that are shared by $\mathcal{D}^E$ and $\mathcal{D}^E_{\text{new}}$, denoted by $\mathcal{D}^\cap = \mathcal{D}^E \cap \mathcal{D}^E_{\text{new}}$.

For any $t \in \mathcal{D}^\cap$, the propagated labels $\widehat{Y}_i(t)$ and $\widehat{Y}_i^{\text{new}}(t)$ are obtained using the same supervision O_i but on different enriched datasets $\mathcal{D}^E$ and $\mathcal{D}^E_{\text{new}}$, respectively. Hence, the propagated label of a shared tuple may change solely due to data updates.

If $\widehat{Y}_i(t) = \widehat{Y}_i^{\text{new}}(t)$, then the propagation error on t remains unchanged. Otherwise, the propagated label flips after the update. By the boundedness of the cost-sensitive error, the change in propagation error on any such tuple is upper-bounded by

$$\left|\ell_{\text{prop}}\big(Y_i(t), \widehat{Y}_i^{\text{new}}(t)\big) - \ell_{\text{prop}}\big(Y_i(t), \widehat{Y}_i(t)\big)\right| \;\le\; \Gamma^{(i)}_{\text{prop}}.$$

As $|\mathcal{D}^\cap| \le |\mathcal{D}^E_{\text{new}}|$, thus, the total increase of propagation error contributed by shared tuples is bounded by

$$\frac{1}{|\mathcal{D}^E_{\text{new}}|} \sum_{t \in \mathcal{D}^\cap} \Gamma^{(i)}_{\text{prop}} \cdot \mathbb{1}\big[\widehat{Y}_i(t) \neq \widehat{Y}_i^{\text{new}}(t)\big]$$

$$\le\; \Gamma^{(i)}_{\text{prop}} \cdot \frac{\sum_{t \in \mathcal{D}^E \cap \mathcal{D}^E_{\text{new}}} \mathbb{1}\big[\widehat{Y}_i(t) \neq \widehat{Y}_i^{\text{new}}(t)\big]}{|\mathcal{D}^E \cap \mathcal{D}^E_{\text{new}}|}.$$

Combining (S1) and (S2) and using the definition of $\Delta^{(i)}$ in Section 6 yields

$$\mathcal{L}^{(i)}_{\text{prop,update}} \;\le\; \mathcal{L}^{(i)}_{\text{prop}} \;+\; \Gamma^{(i)}_{\text{prop}} \cdot \Delta^{(i)}.$$

(S3) **Bounding rewriting error drift.** We next study how the rewriting component of the aggregated error may change. Denote by $\mathcal{L}^{(i)}_{\text{rew}}$ the rewriting error of q^S_i on $\mathcal{D}^E$ and by $\mathcal{L}^{(i)}_{\text{rew,update}}$ the rewriting error on $\mathcal{D}^E_{\text{new}}$, both computed under the same fixed rewrite q^T_i as in SEQR, using the same cost-sensitive rewriting loss instantiated with $\widehat{\pi}^E_i$. As the cost-sensitive rewriting loss is bounded in $[0, \Gamma^{(i)}_{\text{rew}}]$, the rewriting error contribution of newly added tuples is upper-bounded by $\Gamma^{(i)}_{\text{rew}} \cdot \frac{|\mathcal{D}^E_{\text{new}} \setminus \mathcal{D}^E|}{|\mathcal{D}^E_{\text{new}}|}$. For any shared tuple $t \in \mathcal{D}^\cap$, the rewriting loss changes only when the propagated label changes, and the loss change per tuple is bounded by $\Gamma^{(i)}_{\text{rew}}$. Hence,

$$\mathcal{L}^{(i)}_{\text{rew,update}} \;\le\; \mathcal{L}^{(i)}_{\text{rew}} \;+\; \Gamma^{(i)}_{\text{rew}} \cdot \Delta^{(i)}.$$

Finally, since $\mathcal{L}^{(i)}_{\text{update}} = \mathcal{L}^{(i)}_{\text{prop,update}} + \mathcal{L}^{(i)}_{\text{rew,update}}$ and $\mathcal{L}^{(i)}_{\text{opt}} = \mathcal{L}^{(i)}_{\text{prop}} + \mathcal{L}^{(i)}_{\text{rew}}$, combining the bounds in (S1)–(S3) yields

$$\mathcal{L}^{(i)}_{\text{update}} \;\le\; \mathcal{L}^{(i)}_{\text{opt}} \;+\; \big(\Gamma^{(i)}_{\text{prop}} + \Gamma^{(i)}_{\text{rew}}\big) \cdot \Delta^{(i)}.$$

Conditioned on the same high-probability event in Theorem 3 that certifies the static score $\mathcal{L}^{(i)}_{\text{opt}}$ under confidence level δ, the above derivation is deterministic. Therefore, the above inequality holds with probability at least $1 - \delta$, which completes the proof. □

G MORE EXPERIMENTAL RESULTS

Exp-1. Table 6 presents the detailed per-query results summarized in Table 4 for the static setting.

Exp-2. We provide detailed settings and results for our evaluation under dynamic data and query workloads as follows.

Accuracy vs. baselines under dynamic $\mathcal{D}^E$. Table 7 reports the detailed per-query results summarized in Table 5.

Accuracy vs. baselines under dynamic Q^S. Figure 5(a) reports results under *shifted* regime on MMQA, as discussed in Section 7. See our anonymous GitHub (*i.e.,* https://anonymous.4open.science/r/litedbx-C522/) for details on workload construction.

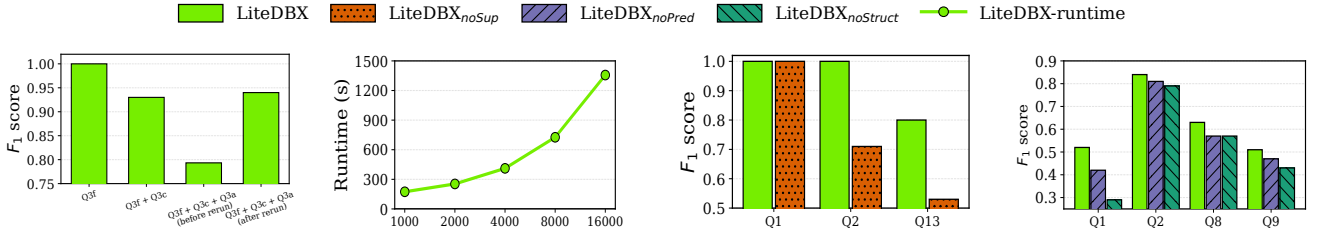
Exp-3. We also assessed LiteDBX via the following tests.

Varying $|\mathcal{D}|$. Following [33], we evaluated the scalability of LiteDBX on Movie by scaling the dataset from 1,000 to 16,000 and running SEQR at each scale. As shown in Figure 5(b), as $|\mathcal{D}|$ increases, latency grows approximately linearly while expense remains nearly unchanged; *e.g.,* from 1,000 to 4,000, runtime cost increases from 173.8s to 411.2s, while the monetary cost stays around $0.0036. This said, LiteDBX remains efficient at larger scales; *e.g.,* it takes 1,356.6s to process 16,000 tuples, at least 4.9× faster than Lotus. This is because (a) larger $\mathcal{D}$ increases the number of scope-filtered tuples to process (*e.g.,* labeling, propagation and attribute population), leading to linear runtime scaling, and (b) LLMs are used only for candidate generation and labeling a small budgeted set (β_{lab}), while large-scale population is handled by offline LLM proxies with low network overhead; in contrast, baselines invoke online LLMs more frequently, incurring higher overhead. Nevertheless, once the schema and SQL rewrites are fixed, LiteDBX achieves substantial cost gains for routine queries (Exp-1). These results validate the scalability of LiteDBX and its scope-aware population strategy.

Varying $\mathcal{U}$. We studied the scalability of LiteDBX *w.r.t.* the complexity of unstructured data $\mathcal{U}$ by starting with image-only inputs and then adding text on E-Comm (not shown). As $\mathcal{U}$ becomes more complex, runtime and monetary cost increase moderately; *e.g.,* from image-only to image+text, runtime (resp. monetary cost) increases from 142.9s (resp. $0.0224) to 178.9s (resp. $0.0236). This is because (a) new modalities introduce additional proxy-based feature extraction and incremental population, but only for scope-filtered tuples; and (b) online LLMs are invoked only for a small and budget-

Datasets	γ	Q^S	LiteDBX			LOTUS			Palimpzest			ThalamusDB			BigQuery			LOTUS+Cache		
			F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)	F_1	$	Time (s)
Movie	0	Q_1	**0.95**	**0.0036**	203.60	0.87	0.5270	629.46	0.88	0.5200	579.59	0.67	0.7000	2590.47	0.62	0.0626	**71.82**	0.86	0.5200	501.68
		Q_2	**1.00**	**0.0027**	36.43	0.72	0.0500	64.85	0.80	0.0070	43.12	0.78	0.0750	287.51	0.60	0.0061	**6.74**	0.67	0.0490	61.25
	33%	Q_1	**0.93**	**0**	**29.65**	0.87	0.6940	769.09	0.88	0.6900	790.16	0.54	0.6900	2345.98	0.75	0.0832	115.38	0.85	0.1600	173.14
		Q_2	**1.00**	**0**	0.28	0.75	0.0500	70.81	0.60	0.0120	57.84	0.83	0.0780	291.85	0.60	0.0061	7.33	0.67	**0**	**0.02**
	66%	Q_1	**0.93**	**0**	**30.11**	0.88	0.8520	992.96	0.88	0.8600	939.53	0.46	0.6800	2262.36	0.85	0.1037	121.24	0.86	0.1700	222.97
		Q_2	**1.00**	**0**	0.29	0.67	0.0500	78.65	0.80	0.0380	89.68	0.78	0.078	263.27	0.60	0.0062	8.48	0.67	**0**	**0.02**
Wildlife	0	Q_1	**1.00**	**0.0050**	**66.56**	**1.00**	0.0650	256.89	**1.00**	0.0400	91.92	**1.00**	0.0600	175.25	**1.00**	0.0655	76.92	**1.00**	0.0650	254.74
	33%	Q_1	**1.00**	**0**	**6.27**	0.86	0.0870	334.79	0.85	0.0500	120.04	0.86	0.0800	258.06	0.85	0.0873	298.40	0.89	0.0220	91.58
	66%	Q_1	**1.00**	**0**	**4.39**	0.89	0.1100	406.98	**1.00**	0.0650	149.07	0.33	0.1000	296.38	**1.00**	0.1093	168.97	0.83	0.0220	89.10
E-Comm.	0	Q_1	**1.00**	**0.0072**	**27.20**	**1.00**	0.5700	545.60	**1.00**	0.4900	455.26	0.73	1.3600	1037.57	**1.00**	0.1283	132.63	0.86	0.5800	523.91
		Q_2	**1.00**	**0.0064**	**21.50**	0.89	0.6800	1168.74	0.56	0.7700	255.38	0.50	0.2900	216.41	0.42	1.5134	1360.36	0.57	0.6900	1167.87
		Q_3	**0.92**	**0.0189**	**110.64**	0.70	0.8300	1469.46	0.58	0.9000	389.65	✗	✗	✗	0.67	0.4489	247.02	0.67	0.8200	1166.05
	33%	Q_1	**1.00**	**0**	**1.05**	0.93	0.7700	743.57	**1.00**	0.6600	531.53	0.73	1.3900	1078.10	**1.00**	0.1709	87.43	0.92	0.1800	175.87
		Q_2	**1.00**	**0**	**0**	0.73	0.9100	1551.15	0.48	1.0200	338.88	0.40	0.2900	236.57	0.45	1.5136	2155.81	0.73	0.2200	374.68
		Q_3	**0.80**	**0**	**7.85**	0.76	1.1000	1587.11	0.56	1.1900	563.89	✗	✗	✗	0.60	0.5953	128.90	0.67	0.2600	376.14
	66%	Q_1	**1.00**	**0**	**0**	**1.00**	0.9800	923.66	**1.00**	0.8300	711.23	0.73	1.3700	1115.02	**1.00**	0.21	221.53	0.92	0.39	369.77
		Q_2	**1.00**	**0**	**0.31**	0.50	1.1500	1946.06	0.50	1.2700	418.23	0.22	0.2880	224.67	0.30	1.5122	1808.68	0.73	0.2400	418.41
		Q_3	**0.80**	**0**	**7.21**	0.70	1.3700	1978.40	0.51	1.5100	889.10	✗	✗	✗	0.58	0.7551	276.19	0.64	0.5500	787.93
MMQA	0	Q_1	0.89	**0.0028**	61.03	0.85	0.0460	62.24	0.86	0.0250	34.11	**0.93**	0.0550	229.36	0.83	0.0093	**1.96**	0.85	0.0460	61.27
		Q_2	**1.00**	**0.0026**	78.38	0.86	0.0490	61.81	0.75	0.0470	54.90	0.85	0.0550	233.12	0.60	0.0092	**1.58**	0.86	0.0490	55.91
		Q_3	**1.00**	**0.0013**	**10.59**	0.20	0.0500	69.90	**1.00**	0.019	28.61	0	0.1300	801.23	**1.00**	0.0057	11.50	0.29	0.0500	61.49
		Q_4	**1.00**	**0.0015**	**8.19**	0.29	0.0480	76.39	**1.00**	0.0370	40.47	0.13	0.1270	585.81	**1.00**	0.0058	10.08	0.33	0.0558	55.94
		Q_5	**1.00**	**0.0013**	13.02	0.42	0.0610	121.39	**1.00**	0.0200	29.57	0.17	0.1500	868.18	0.56	0.0058	**9.95**	0.53	0.0608	95.49
	33%	Q_1	0.87	**0**	5.20	0.85	0.0620	90.06	0.86	0.0340	34.12	**0.90**	0.0760	347.80	0.87	0.0123	**3.15**	0.85	0.0160	31.23
		Q_2	**1.00**	**0**	5.51	**1.00**	0.0630	80.31	0.86	0.0630	77.26	0.86	0.0740	297.81	0.55	0.0122	**2.35**	0.86	0.0150	20.21
		Q_3	**1.00**	**0**	**0.74**	0.13	0.0720	95.80	**1.00**	0.0260	35.21	0	0.1500	950.88	**1.00**	0.0077	11.70	0.22	0.0170	27.70
		Q_4	**1.00**	**0**	**0.74**	0.15	0.0660	93.97	0.50	0.0500	73.23	0.08	0.1500	874.61	**1.00**	0.0077	12.31	0.29	0.0210	48.68
		Q_5	**1.00**	**0**	**1.61**	0.42	0.0770	133.36	**1.00**	0.0270	37.97	0.18	0.1900	910.01	0.59	0.0077	10.06	0.45	0.0190	35.97
	66%	Q_1	0.81	**0**	5.56	0.73	0.0770	95.37	0.80	0.0400	43.70	0.80	0.1000	503.26	0.80	0.0151	**2.46**	**0.85**	0.0150	23.00
		Q_2	**1.00**	**0**	5.15	0.86	0.0800	104.04	0.86	0.0780	94.00	**1.00**	0.1000	370.85	0.60	0.0152	**4.14**	0.86	0.0160	26.16
		Q_3	**1.00**	**0**	**0.68**	0.25	0.0870	114.67	**1.00**	0.0320	54.14	0.08	0.1900	1101.00	**1.00**	0.0096	18.90	0.22	0.0160	31.35
		Q_4	**1.00**	**0**	**0.69**	0.22	0.0880	130.14	**1.00**	0.0630	87.99	0.08	0.2400	1237.96	**1.00**	0.0095	17.30	0.29	0.0160	26.23
		Q_5	**1.00**	**0**	**1.15**	0.42	0.1000	151.44	**1.00**	0.0330	52.36	0.07	0.2400	1062.16	0.63	0.0097	17.4	0.43	0.0190	37.39
Medical	0	Q_1	**0.44**	**0.0012**	124.90	0.20	0.3400	457.13	0.13	0.1800	222.71	0.17	0.5900	1784	0.10	0.0468	**15.69**	0.10	0.3600	377.44
		Q_2	**0.83**	**0.0022**	419.60	0.48	0.1000	195.78	0.52	0.1590	172.26	0.17	0.0740	298.26	0.64	2.7591	1117.91	0.42	0.1000	**116.63**
		Q_3	**0.57**	**0.0019**	112.09	0.22	0.0320	52.59	0.32	0.0630	79.86	0.25	0.0360	333.21	0.43	0.3598	463.88	0.24	0.0310	**46.60**
		Q_4	**0.50**	**0.0037**	423.60	0	0.1200	180.00	0.20	0.1700	98.36	TO	TO	TO	0.39	0.1749	**65.48**	0	0.1100	173.51
	33%	Q_1	**0.53**	**0.0043**	172.84	0.20	0.4600	614.50	0.24	0.2300	292.89	0.22	0.7300	2084.25	0.12	0.0465	**15.64**	0.25	0.1900	213.35
		Q_2	**0.87**	**0**	112.56	0.63	0.1300	253.71	0.70	0.2100	239.02	0.16	0.0740	284.61	0.75	2.7591	1340.52	0.68	0.0600	**71.30**
		Q_3	0.51	**0**	**24.84**	0.30	0.0430	67.08	**0.62**	0.0800	104.55	0.25	0.0360	300.63	0.54	0.3598	459.30	0.32	0.0180	35.06
		Q_4	0.47	**0**	61.90	0.01	0.1500	229.94	0.26	0.2200	132.88	TO	TO	TO	**0.48**	0.2303	**43.16**	0	0.0610	97.59
	66%	Q_1	**0.52**	**0**	**22.16**	0.33	0.5800	683.00	0.26	0.7000	689.12	0.32	0.8200	2350.88	0.19	0.0465	59.01	0.32	0.0410	52.69
		Q_2	0.86	**0**	104.61	0.79	0.1670	307.41	**0.88**	0.2680	271.33	0.19	0.0730	292.17	0.85	2.7591	1103.59	0.79	0.0100	**21.76**
		Q_3	0.50	**0**	23.25	0.36	0.0520	77.50	0.61	0.1023	115.85	0.28	0.0360	311.81	**0.63**	0.3598	484.13	0.35	0.0040	**10.57**
		Q_4	0.46	**0**	59.77	0	0.1800	285.29	0.31	0.2800	176.28	TO	TO	TO	**0.56**	0.2900	98.14	0	0.0150	**31.13**

Table 7: Performance comparison of LiteDBX and baselines on dynamic data workloads. Bold (resp. underlined) entries indicate the best (resp. second-best) results on full datasets; ✗ (resp. TO) denotes unsupported workload (resp. timeout after 1 hour).



(a) MMQA: varying Q^S (F_1) (b) Movie: varying $\mathcal{D}$ (time) (c) E-Comm: supervision impact (F_1) (d) Medical: conf. metrics impact (F_1)

Figure 5: Performance evaluation

bounded set of sampled tuples, while most attribute population is handled by offline proxies. These results demonstrate the scalability of LiteDBX across heterogeneous $\mathcal{U}$.

Varying $|Q^S|$. We evaluated the scalability of LiteDBX by increasing the semantic query workload Q^S from 1 to 3 distinct queries on E-Comm (not shown). When $|Q^S|$ grows from 1 to 3, monetary cost increases almost linearly, *e.g.,* from 0.0162$ to 0.0404$, since each query incurs its own budget-bounded labeling and attribute suggestion with online LLM. In contrast, runtime increases only slightly (from 122.6s to 138.4s), since one-time enrichment results are reused to support different queries. This shows LiteDBX's ability to efficiently support multiple semantically diverse queries.

Ablation. Figure 5(c) presents results on the impact of human-supervised coreset selection on E-Comm, as discussed in Section 7.

More ablation. We also evaluated LiteDBX via following tests.

Impact of confidence metrics. We studied the role of confidence metrics using two variants, LiteDBX$_{\text{noStruct}}$ and LiteDBX$_{\text{noPred}}$, which remove the structural and prediction confidence terms, respectively, in coreset construction on Medical. As shown in Figure 5(d), on average the F_1 of LiteDBX is 0.625, 20.2% and 10.1% higher than LiteDBX$_{\text{noStruct}}$ and LiteDBX$_{\text{noPred}}$, up to 79.3% and 23.8%, respectively. This is because prediction confidence leverages human supervision to prioritize high-reliability samples, while structural confidence regularizes coreset selection to mitigate overfitting under limited labels. These results validate our coreset construction strategy.

Benefits of incremental maintenance. We assessed the incremental

maintenance strategy under evolving data by comparing LiteDBX with a variant, LiteDBX$_{\text{noMain}}$, which reruns SEQR from scratch at each scale in Exp-2 (not shown). LiteDBX substantially reduces monetary and time cost with only marginal accuracy loss, *e.g.,* 94.4% and 69.3% reductions in monetary and time cost, respectively, with only a 1.9% drop in F_1. These gains stem from: (a) partial population of $\mathcal{D}^E$ for affected tuples only, (b) incremental updates of coresets and pseudo labels, and (c) reuse of rewrites via lightweight certificates, triggering refresh only when reuse becomes unsafe. These results validate incremental maintenance for evolving workloads.

Impact of error metrics. We evaluated the role of error metrics using two variants, LiteDBX$_{\text{subj}}$ (resp. LiteDBX$_{\text{obj}}$), which uses the subjective (resp. objective) error term only, when computing SEQR's static error (not shown). On average the F_1 of LiteDBX is 0.625, 28.9% and 17.9% higher than LiteDBX$_{\text{subj}}$ and LiteDBX$_{\text{obj}}$, up to 136.4% and 126.1%, respectively. This is because these error metrics capture complementary aspects of SEQR (*i.e.,* propagation reliability and rewrite fidelity), both essential for accurate error estimation. These results validate our error-metric design.